

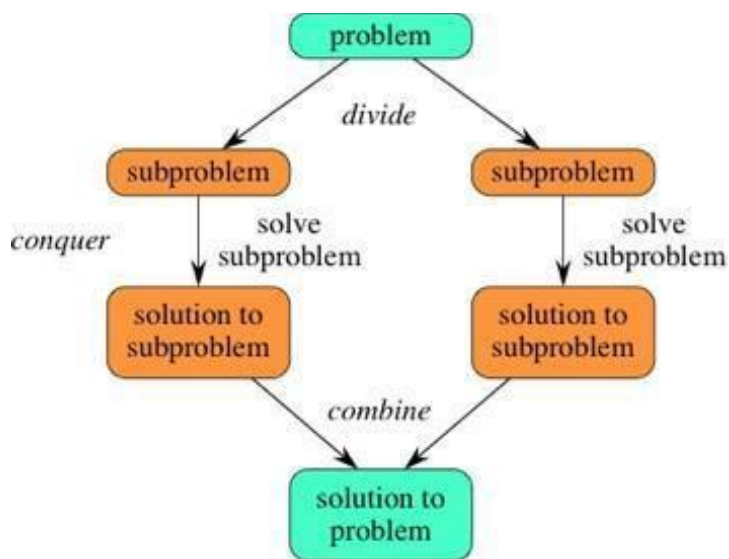
MODULE III

DIVIDE AND CONQUER

Top-down approach for problem solving. Designs an algorithm that divides the problem into small subproblems. Solutions of subproblems are easier to find and then combines the partial solutions to get the original solution.

Divide and conquer algorithm is having three parts:

1. **Divide** the problem into a number of sub-problems that are smaller instances of the same problem.
2. **Conquer** the sub-problems by solving them recursively. If they are small enough, solve the sub-problems as base cases.
3. **Combine** the solutions to the sub-problems into the solution for the original problem.



- **Control Abstraction:** It is a procedure whose flow of control is clear but whose primary operations are specified by other procedure whose precise meanings are left undefined.
- **Control Abstraction: Divide and Conquer**

Algorithm DAndC(P)

```
{
    if Small(P) then
        return S(P)
    else
    {
        Divide P into smaller instances  $P_1, P_2, \dots, P_k, k \geq 1$ ;
        apply DAndC to each of these sub-problems;
        return Combine(DAndC( $P_1$ ), DAndC( $P_2$ ),....., DAndC( $P_k$ ));
    }
}
```

- If the given problem is small, return the result
- Otherwise, divide the problem into smaller instances $P_1, P_2, \dots, P_k$
- Apply DAndC() to each of these sub-problems.
- Finally combine the results of all sub-problems.

- DAndC() can be described using the following recurrence relation:

$$T(n) = \begin{cases} g(n) & n \text{ is small} \\ T(n_1) + T(n_2) + \dots + T(n_k) + f(n) & \text{Otherwise} \end{cases}$$

- $T(n)$: Time for divide and conquer on any input of size n
- $f(n)$: Complexity of dividing the problem and combining the results.
- Complexity of many divide and conquer algorithms are given by the following recurrence relation

$$T(n) = \begin{cases} T(1) & n = 1 \\ aT(n/b) + f(n) & n > 1 \end{cases}$$

where a and b are constants. We assume $T(1)$ is known and n is power of b (ie: $n=b^k$)

2-way Merge Sort

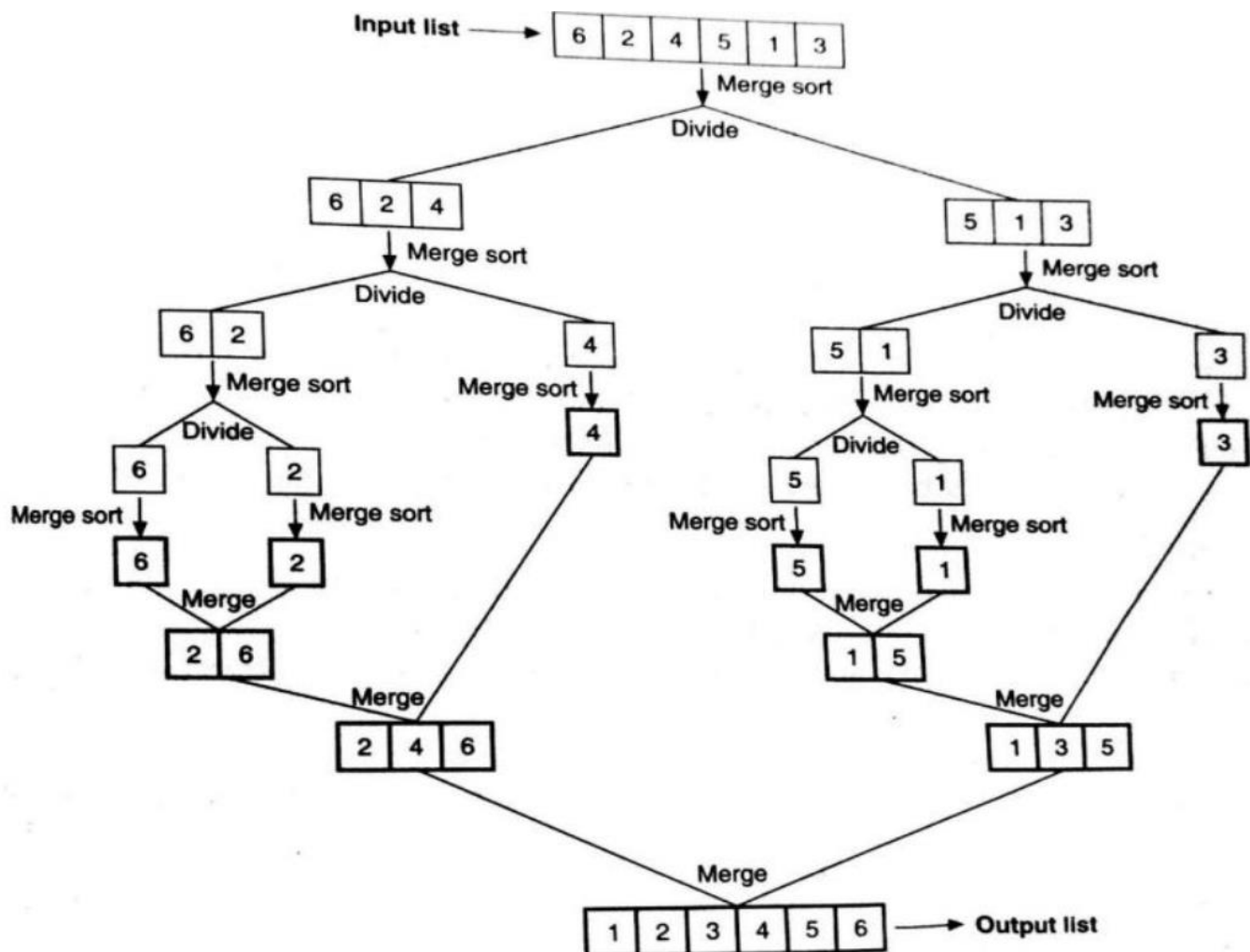


Figure 10.57 Illustration of the merge sort.

MERGE SORT

Given a sequence of n elements (also called keys) $a[1], \dots, a[n]$, they are split into two sets $a[1], \dots, a[n/2]$ and $a[n/2+1], \dots, a[n]$. Each set is individually sorted and the resulting sorted sequences are merged to produce a single sorted sequence of n elements.

Merge sort on an input array with n elements

Divide :- Partition array into two sublists s_1 and s_2 with $n/2$ elements each.

Conquer :- Sort sublist s_1 and sublist s_2

Combine :- Merge s_1 and s_2 into a unique sorted group.

In merge sort, division of array is based on the positions of array elements.

Algorithm MergeSort (low, high)

```

1. Algorithm MergeSort (low, high)
2. // a[low:high] is a global array to be sorted.
3. // Small(P) is true if there is only one element.
4. // to sort. In this case, the list is already sorted.
5. {
6.   if (low < high) then // if there are more than one
7.     // Divide P into subproblems
8.     // Find where to split the set
9.     mid := (low + high) / 2;
10.    // solve subproblems
11.    MergeSort (low, mid);
12.    MergeSort (mid + 1, high);

```

```

14 //Combine the solutions .
15 Merge (low, mid, high);
16 }
17 }

```

Algorithm Merge to merge two sorted subarrays using auxillary storage.

```

1. Algorithm Merge (low, mid, high)
2. //a[low: high] is a global array containing two
3. //sorted subsets in a[low: mid] and in
4. //a[mid+1: high]. The goal is to merge these
5. //two sets into a single set residing in
6. //a[low: high]. b[] is an auxillary global array.
7. {
8.   h := low; i := low; j := mid+1;
9.   while ((h ≤ mid) and (j ≤ high)) do
10.    {
11.      if (a[h] ≤ a[j]) then
12.        {
13.          b[i] := a[h]; h := h+1;
14.        }
15.      else
16.        {
17.          b[i] := a[j]; j := j+1;
18.        }
19.      i := i+1;
20.    }
21.   if (h > mid) then
22.     for k := j to high do
23.       {

```


$b[i] := a[k]; \quad i := i+1;$

}

else

for $k := h$ to mid do

{

$b[i] := a[k]; \quad i := i+1;$

}

for $k := \text{low}$ to high do $a[k] := b[k]$

$a[k] := b[k];$

}

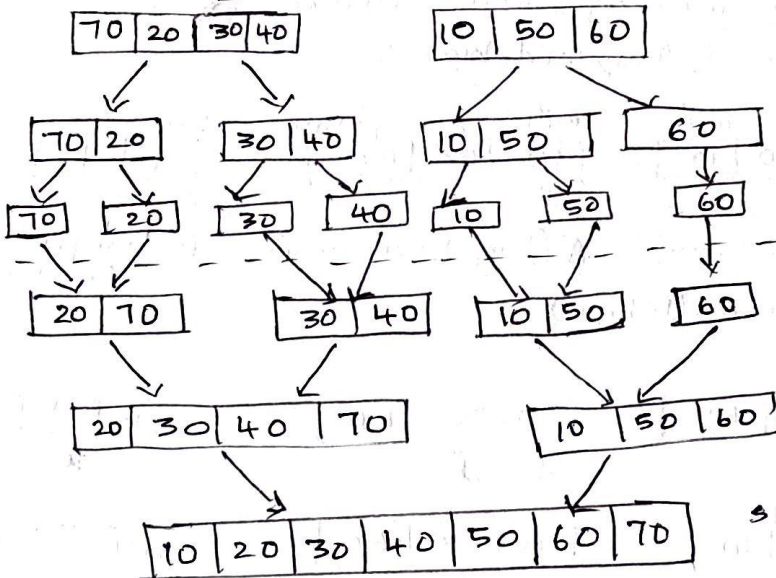
Example:-

Sort 70, 20, 30, 40, 10, 50, 60 using Merge Sort.

1 2 3 4 5 6 7
70 | 20 | 30 | 40 | 10 | 50 | 60

Unsorted List.

Divide.



sort & Merge.

sorted List.

Working:-

Divide
Merge Sort
(low, high)

Position $\rightarrow$

Elements $\rightarrow$

$$\text{mid} = (1 + 7) / 2 = 4$$

$$\text{mid} = (1 + 4) / 2 = 2$$

$$\text{mid} = (5 + 7) / 2 = 6$$

$$\text{mid} = (1 + 2) / 2 = 1$$

$$\text{mid} = (3 + 4) / 2 = 3$$

$$\text{mid} = (5 + 6) / 2 = 5$$

Merge
(low, mid, high)

[Sorting and merging]

Merge (1, 2), (3, 4), (5, 6)

Merge (1, 2, 3, 4), (5, 6, 7)

Merge (1, 2, 3, 4, 5, 6, 7)

a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]
70	20	30	40	10	50	60
70	20	30	40	10	50	60
70	20	30	40	10	50	60
70	20	30	40	10	50	60
20	70	30	40	10	50	60
20	30	40	70	10	50	60
10	20	30	40	50	60	70

Working of combining two subarrays using

Algorithm Merge (low, mid, high)

low	2	3	mid
20	30	40	70
h			

A (Left subarray)

mid+1	6	high
10	50	60
j		

A (Right subarray)

low	high
i	

B (temporary array)
(to merged)

While ($h \leq \text{mid}$) and ($i \leq \text{high}$)

≡

20	30	40	70
h			

10	50	60
j		

10					
i					

20	30	40	70
h			

10	50	60
j		

10	20				
i					

20	30	40	70
h			

10	50	60
j		

10	20	30			
i					

20	30	40	70
h			

10	50	60
j		

10	20	30	40		
i					

20	30	40	70
----	----	----	----

h

10	50	60
----	----	----

j^o

10	20	30	40	50		
----	----	----	----	----	--	--

1^o

20	30	40	70
----	----	----	----

h

10	50	60
----	----	----

j^o

10	20	30	40	50	60	
----	----	----	----	----	----	--

1^o

if ($h > mid$)

20	30	40	70
----	----	----	----

A (Left sublist)

10	50	60
----	----	----

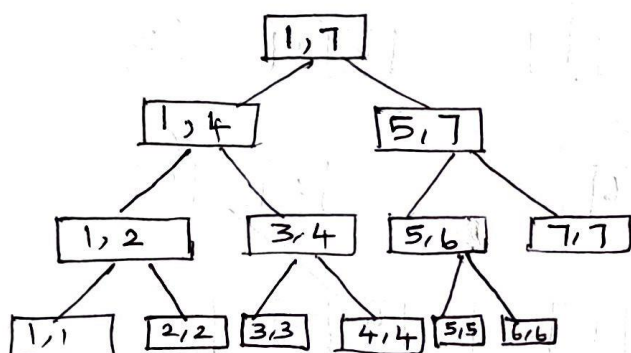
A (Right sublist)

10	20	30	40	50	60	70
----	----	----	----	----	----	----

B (Sorted List)

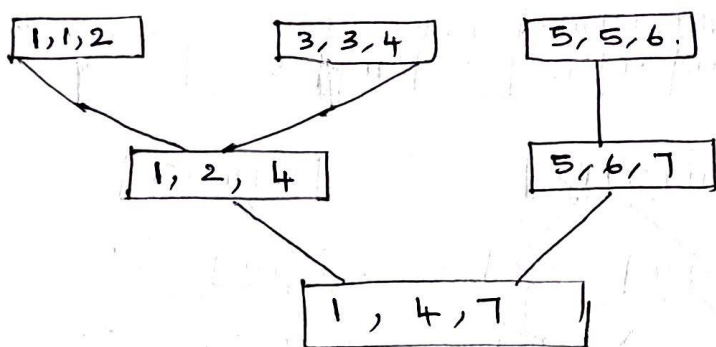
Tree of Call MergeSort(1, 7)

[MergeSort (low, high)]



Tree of Call Merge(1, 4, 7)

[Merge (low, mid, high)]

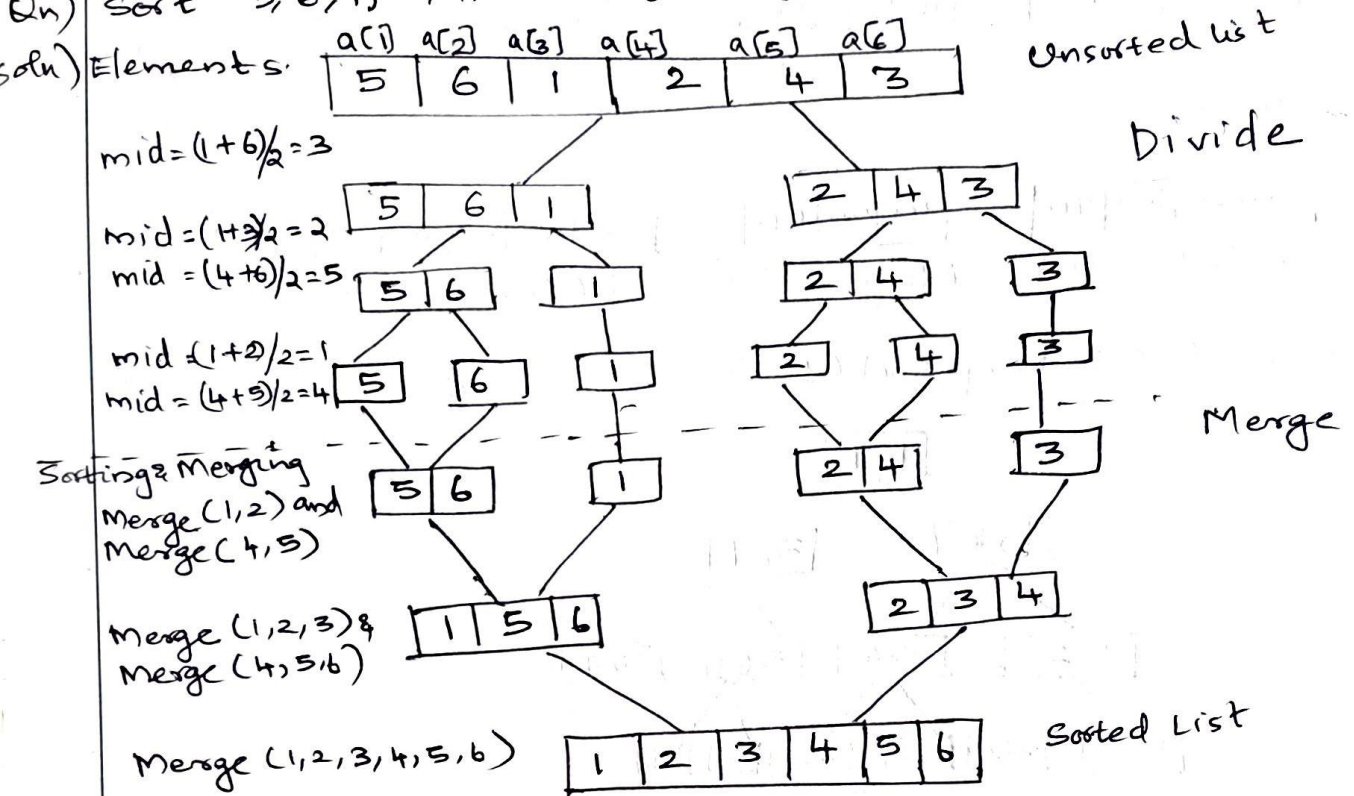


Working
(See the Table
and draw)

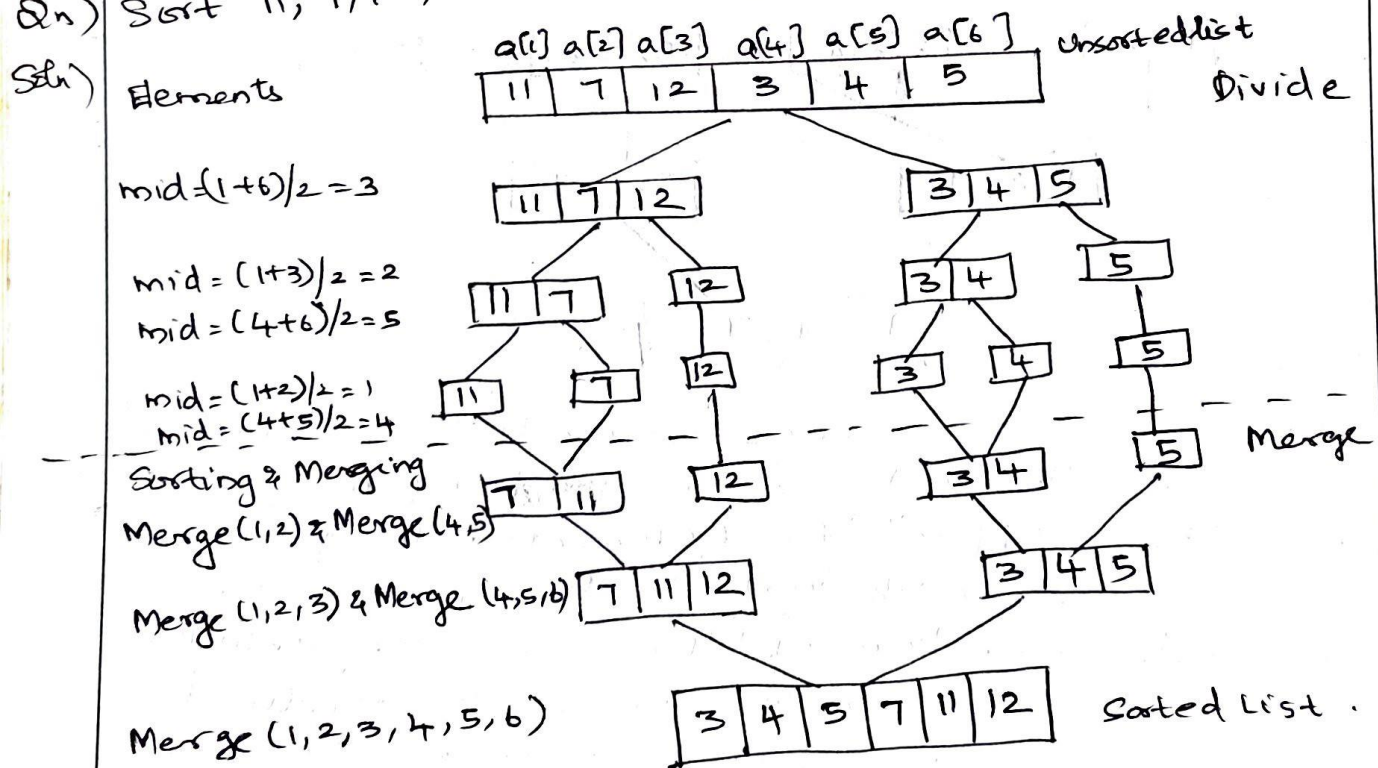
Fig (1) is the tree, which represents the sequence of recursive calls that are produced by mergesort when it is applied to 7 elements. The pair of values in each row is the parameter low and high. Here splitting continues until the set containing single element is produced. Fig (2) represents sequence of recursive calls that are produced by merge algorithm. Here,

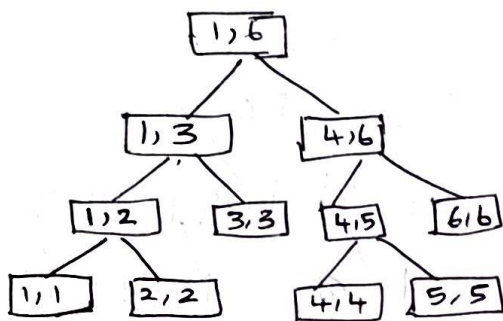
the parameters are low, mid, high.

Qn) Sort 5, 6, 1, 2, 4, 3. using Merge sort

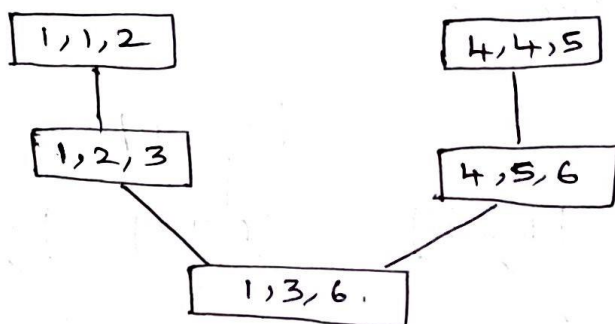


Qn) Sort 11, 7, 12, 3, 4, 5 using Merge sort





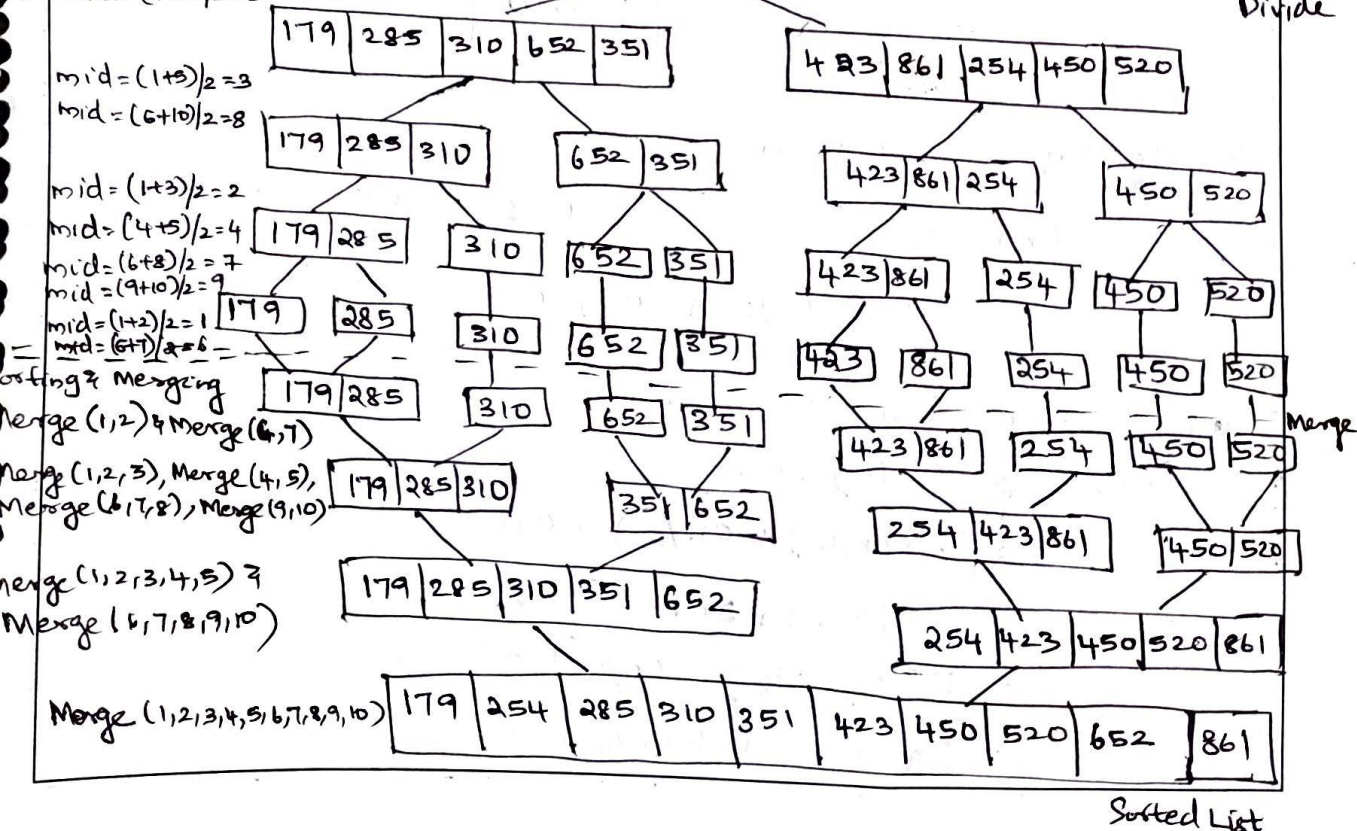
{ Merge(Low, mid, high) }



Elements

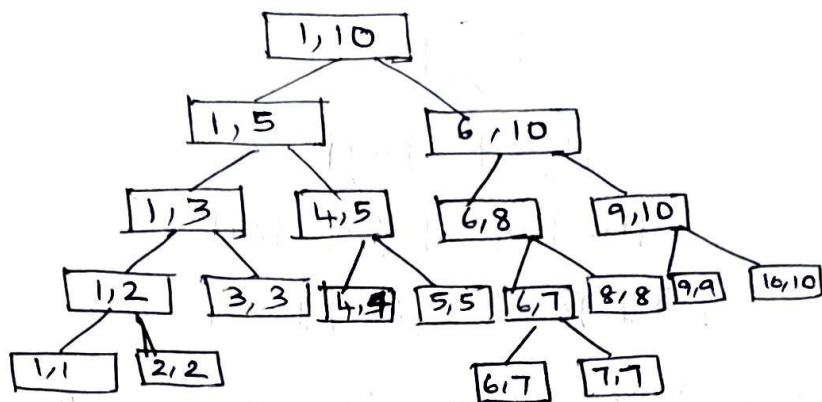
$a[1]$	$a[2]$	$a[3]$	$a[4]$	$a[5]$	$a[6]$	$a[7]$	$a[8]$	$a[9]$	$a[10]$
179	285	310	652	351	423	861	254	450	520

Unsorted list

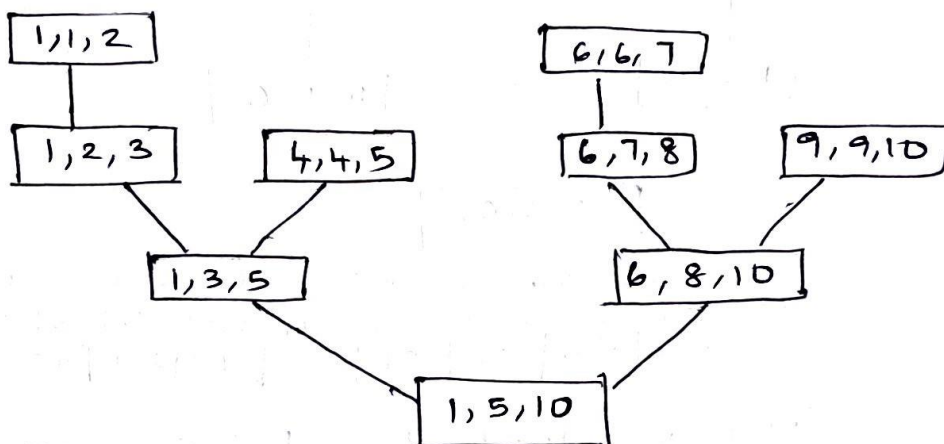
$$\text{mid} = (1+10)/2 = 5$$


Sorted List

Tree of call Mergesort (1,10)



Tree of call for Merge (1,5,10)



Complexity of Mergesort

$$T(n) = \begin{cases} a, & n=1 \\ 2T(n/2) + cn, & n>1 \end{cases}$$

Where a, c are constants.

$$\begin{aligned}
 T(n) &= 2T(n/2) + cn \\
 &= 2[2T(n/4) + cn/2] + cn \\
 &= 4T(n/4) + 2cn/2 + cn \\
 &= 4[2T(n/8) + cn/4] + 2cn \\
 &= 8T(n/8) + 4cn/4 + 2cn
 \end{aligned}$$

$$= 8T(n/8) + 3cn$$

$$= 2^3 T(n/2^3) + 3cn$$

$$\vdots$$

$$= 2^k T(n/2^k) + kcn$$

$$2^k = n$$

$$k = \log n$$

$$= n \cdot a + \log n \cdot cn$$

$$T(n) = an + cn \cdot \log n$$

$$\therefore \text{Complexity} = \underline{\underline{O(n \log n)}}$$

// Best & Worst Case

Computing time for merge sort is described in the recurrence relation. n is the power of 2. i.e., $n = 2^k$.

Divide and Conquer Matrix Multiplication

Let A and B be two $n \times n$ matrices. The product $C = AB$ is also an $n \times n$ matrix whose i, j th element is formed by taking the elements in the i th row of A and the j th column of B and multiplying them to get,

$$C(i, j) = \sum_{1 \leq k \leq n} A(i, k) \cdot B(k, j)$$

$$\forall i, j < n$$

To compute $C(i, j)$ using this formula, n multiplications are required. As the matrix C has n^2 elements, time for resulting matrix multiplication algorithm is $O(n^3)$.

Divide and Conquer method to Compute ^{two} $n \times n$ matrices requires to assume that n is a power of 2. i.e., there exists a non-negative integer k such that $n = 2^k$. If n is not a power of two, then enough rows and columns of zeros can be added to both A and B , so that resulting dimensions are power of 2.

Imagine A and B are each partitioned into four square submatrices, each submatrix having dimensions $n/2 \times n/2$. Then, the product AB can be computed by using the above formula for the product of 2×2 matrices. If AB is

$$\begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix} = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} \quad \text{--- ①}$$

then,

$$\left. \begin{aligned} C_{11} &= A_{11}B_{11} + A_{12}B_{21} \\ C_{12} &= A_{11}B_{12} + A_{12}B_{22} \\ C_{21} &= A_{21}B_{11} + A_{22}B_{21} \\ C_{22} &= A_{21}B_{12} + A_{22}B_{22} \end{aligned} \right\} \quad \text{--- ②}$$

This ①, ② are computed for $n=2$.

For $n > 2$, C can be computed using matrix multiplication and addition operations applied to matrices of size $n/2 \times n/2$. Since, n is a power of 2, these matrix products can be recursively computed by the same algorithm; until $n=2$, so that product can be computed directly.

To compute AB using ②, we require

① 8 multiplication

② 4 additions

of $n/2 \times n/2$ matrices.

Since two $n/2 \times n/2$ matrices can be added in time cn^2 for some constant c , the overall computing time $T(n)$ of the resulting divide and conquer algorithm is given by the recurrence.

$$T(n) = \begin{cases} b & , n \leq 2 \\ 8T(n/2) + cn^2 & , n > 2 \end{cases}$$

where, b and c are constants.

[$8T(n/2)$ is time for multiplication,
 cn^2 is time for addition]

Analysis of Matrix Multiplication

$$T(n) = 8T(n/2) + cn^2$$

Substitute 2^k for n ,

$$T(2^k) = 8T\left(\frac{2^k}{2}\right) + c \cdot 2^{2k}$$

$$T(2^k) = 8T(2^{k-1}) + c \cdot 2^{2k}$$

Substitute $T(k)$ for $T(2^k)$

$$T(k) = 8T(k-1) + c \cdot 2^{2k} \quad \text{--- ①}$$

$$T(k-1) = 8T(k-1-1) + c \cdot 2^{2(k-1)}$$

$$T(k-1) = 8T(k-2) + c \cdot 2^{2k} \cdot 2^{-2} \quad \text{--- ②}$$

Multiply eq ② $\times 4$.

$$4T(k-1) = 32T(k-2) + c \cdot 2^{2k} \cdot 2^{-2} \cdot 2^2$$

$$4T(k-1) = 32T(k-2) + c \cdot 2^{2k} \quad \text{--- ③}$$

eq ① - eq ③,

$$T(k) = 8T(k-1) + c \cdot 2^{2k}$$

$$4T(k-1) = 32T(k-2) + c \cdot 2^{2k}$$

$$T(k) - 4T(k-1) = 8T(k-1) - 32T(k-2)$$

$$T(k) - 4T(k-1) - 8T(k-1) = -32T(k-2)$$

$$T(k) - 12T(k-1) + 32T(k-2) = 0$$

Put x^k for $T(k)$

$$x^k - 12x^{k-1} + 32x^{k-2} = 0$$

Put $k=2$ or $k \geq 2$

$$x^2 - 12x + 32 = 0$$

$$(x_1 - 8)(x_2 - 4) = 0$$

$$x_1 = 8, x_2 = 4$$

$$\begin{aligned} T(n) &= C_1 8^k + C_2 4^k \\ &= C_1 (2^k)^3 + C_2 (2^k)^2 \\ &= C_1 n^3 + C_2 n^2 \end{aligned}$$

$$\therefore \underline{T(n) = O(n^3)} \Rightarrow \text{Complexity.}$$

The recurrence has an order of n^3 , because matrix multiplication require more multiplications compared to additive terms. $\therefore$ Matrix multiplication should be performed with fewer multiplication and larger addition/subtractions.

Strassen's Matrix Multiplication:-

Strassen Volker discovered a way to compute C_{ij} 's using only ① 7 multiplications and ② 18 additions/subtractions.

This method involve computing 7 $n/2 \times n/2$ matrices $P_1, P_2, P_3, P_4, P_5, P_6, P_7$. Then C_{ij} 's are computed. $P_1, P_2, P_3, P_4, P_5, P_6, P_7$ require 7 matrix multiplications and 10 matrix additions/subtractions. The C_{ij} 's r, s, t, u require an additional 8 additions/subtractions.

C

=

A

B

$$\begin{bmatrix} r & s \\ t & u \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} e & f \\ g & h \end{bmatrix}$$

$$P_1 = a * (f - h)$$

$$P_2 = (a + b) * h$$

$$P_3 = (c + d) * e$$

$$P_4 = d * (g - e)$$

$$P_5 = (a + d) * (e + h)$$

$$P_6 = (b - d) * (g + h)$$

$$P_7 = (a - c) * (e + f)$$

$$r = P_5 + P_4 - P_2 + P_6$$

$$s = P_1 + P_2$$

$$t = P_3 + P_4$$

$$u = P_5 + P_1 - P_3 - P_7$$

The resulting recurrence relation for $T(n)$ is,

$$T(n) = \begin{cases} b & , n \leq 2 \\ 7T(n/2) + cn^2 & , n > 2 \end{cases}$$

where b, c are constants.

Analysis of Strassen's Matrix Multiplication:-

$$T(n) = 7T(n/2) + cn^2$$

Substitute 2^k for n ,

$$T(2^k) = 7T\left(\frac{2^k}{2}\right) + c \cdot 2^{2k}$$

$$T(2^k) = 7T(2^{k-1}) + c \cdot 2^{2k}$$

Substitute $T(k)$ for $T(2^k)$

$$T(k) = 7T(k-1) + c \cdot 2^{2k} \quad \text{--- (1)}$$

$$T(k-1) = 7T(k-1-1) + c \cdot 2^{2(k-1)}$$

$$T(k-1) = 7T(k-2) + c \cdot 2^{2k} \cdot 2^{-2} \quad \text{--- (2)}$$

Multiply (2) x 4

$$4T(k-1) = 28T(k-2) + C \cdot 2^2 \cdot 2^{2k} \cdot 2^{-2}$$

$$4T(k-1) = 28T(k-2) + C \cdot 2^{2k} \quad \text{--- (3)}$$

eq ① - eq ③

$$T(k) = 7T(k-1) + C \cdot 2^{2k}$$

$$4T(k-1) = 28T(k-2) + C \cdot 2^{2k}$$

$$T(k) - 4T(k-1) = 7T(k-1) - 28T(k-2)$$

$$T(k) - 11T(k-1) + 28T(k-2) = 0$$

Put x^k for $T(k)$

$$x^k - 11x^{k-1} + 28x^{k-2} = 0$$

Put $k=2$

$$x^2 - 11x + 28 = 0$$

$$(x_1 - 7)(x_2 - 4) = 0$$

$$\therefore x_1 = 7, x_2 = 4$$

$$T(n) = C_1 7^k + C_2 4^k$$

$$= C_1 7^{\log n} + C_2 (2^k)^2$$

$$= C_1 n^{\log 7} + C_2 n^2$$

$$= C_1 n^{(2.81)} + C_2 n^2$$

$$\therefore T(n) = O(n^{2.81})$$

$$\left\{ a^{\log_b c} = c^{\log_b a} \right\}$$

$$\left[n = 2^k \right]$$

$$\log n = \log 2^k$$

$$\log n = k \log 2$$

$$\log n = k$$

$$(\log 2 = 1)$$

Examples:-

$$A = \begin{bmatrix} 7 & 9 \\ 2 & 5 \end{bmatrix}$$

$$B = \begin{bmatrix} 3 & 2 \\ 6 & 5 \end{bmatrix}$$

$$C = A \cdot B$$

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

$$B = \begin{bmatrix} e & f \\ g & h \end{bmatrix}$$

$$P_1 = a * (f - b) = 7 * (2 - 5) = -21$$

$$P_2 = (a + b) * h = (7 + 9) * 5 = 80$$

$$P_3 = (c + d) * e = (2 + 5) * 3 = 21$$

$$P_4 = d * (g - e) = 5 * (6 - 3) = 15$$

$$P_5 = (a + d) * (e + h) = (7 + 5) * (3 + 5) = 96$$

$$P_6 = (b - d) * (g + h) = (9 - 5) * (6 + 5) = 44$$

$$P_7 = (a - c) * (e + f) = (7 - 2) * (3 + 2) = 25$$

$$r = P_5 + P_4 - P_2 + P_6 = 96 + 15 - 80 + 44 = 75$$

$$s = P_1 + P_2 = -21 + 80 = 59$$

$$t = P_3 + P_4 = 21 + 15 = 36$$

$$u = P_5 + P_1 - P_3 - P_7 = 96 - 21 - 21 - 25 = 29$$

$$\therefore C = \begin{bmatrix} r & s \\ t & u \end{bmatrix} = \begin{bmatrix} 75 & 59 \\ 36 & 29 \end{bmatrix}$$

Qn) Solve, using Strassen's Algorithm to compute matrix product.

$$A = \begin{bmatrix} 1 & 3 \\ 7 & 5 \end{bmatrix}, \quad B = \begin{bmatrix} 6 & 8 \\ 4 & 2 \end{bmatrix}$$

Soln)

$$P_1 = 1 * (8 - 2) = 6$$

$$P_2 = (1 + 3) * 2 = 8$$

$$P_3 = (7 + 5) * 6 = 72$$

$$P_4 = 5 * (4 - 6) = -10$$

$$P_5 = (1 + 5) * (6 * 2) = 48$$

$$P_6 = (3 - 5) * (4 + 2) = -12$$

$$P_7 = (1 - 7) * (6 + 8) = -84$$

$$r = 48 - 10 - 8 - 12 = 18$$

$$s = 6 + 8 = 14$$

$$t = 72 - 10 = 62$$

$$u = 48 + 6 - 72 + 84 = 66$$

$$\therefore C = \begin{bmatrix} r & s \\ t & u \end{bmatrix} = \begin{bmatrix} 18 & 14 \\ 62 & 66 \end{bmatrix}$$

Solve using Strassen's Multiplication

$$A = \begin{bmatrix} \textcircled{a} & & \textcircled{b} & \\ 4 & 2 & 0 & 1 \\ 3 & 1 & 2 & 5 \\ \hline & & & \\ 3 & 2 & 1 & 4 \\ 5 & 2 & 6 & 7 \\ \textcircled{c} & & \textcircled{d} & \end{bmatrix}$$

$$B = \begin{bmatrix} \textcircled{e} & & \textcircled{f} & \\ 2 & 1 & 3 & 2 \\ 5 & 4 & 2 & 3 \\ \hline & & & \\ 1 & 4 & 0 & 2 \\ 3 & 2 & 4 & 1 \\ \textcircled{g} & & \textcircled{h} & \end{bmatrix}$$

$$P_1 = a * (f - h)$$

$$= \begin{bmatrix} 4 & 2 \\ 3 & 1 \end{bmatrix} * \left[\begin{bmatrix} 3 & 2 \\ 2 & 3 \end{bmatrix} - \begin{bmatrix} 0 & 2 \\ 4 & 1 \end{bmatrix} \right]$$

$$= \begin{bmatrix} 4 & 2 \\ 3 & 1 \end{bmatrix} * \begin{bmatrix} 3 & 0 \\ -2 & 2 \end{bmatrix}$$

$$P_1 = \begin{bmatrix} 12-4 & 0+4 \\ 9-2 & 0+2 \end{bmatrix} = \underline{\underline{\begin{bmatrix} 8 & 4 \\ 7 & 2 \end{bmatrix}}}$$

$$P_2 = (a+b) * h$$

$$= \left[\begin{bmatrix} 4 & 2 \\ 3 & 1 \end{bmatrix} + \begin{bmatrix} 0 & 1 \\ 2 & 5 \end{bmatrix} \right] * \begin{bmatrix} 0 & 2 \\ 4 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} 4 & 3 \\ 5 & 6 \end{bmatrix} * \begin{bmatrix} 0 & 2 \\ 4 & 1 \end{bmatrix}$$

$$P_2 = \underline{\underline{\begin{bmatrix} 12 & 11 \\ 24 & 16 \end{bmatrix}}}$$

$$P_3 = (c+d) * e$$

$$= \left[\begin{bmatrix} 3 & 2 \\ 5 & 2 \end{bmatrix} + \begin{bmatrix} 1 & 4 \\ 6 & 7 \end{bmatrix} \right] * \begin{bmatrix} 2 & 1 \\ 5 & 4 \end{bmatrix}$$

$$= \begin{bmatrix} 4 & 6 \\ 11 & 9 \end{bmatrix} * \begin{bmatrix} 2 & 1 \\ 5 & 4 \end{bmatrix} = \underline{\underline{\begin{bmatrix} 38 & 28 \\ 67 & 47 \end{bmatrix}}}$$

$$P_4 = d * (g - e)$$

$$= \begin{bmatrix} 1 & 4 \\ 6 & 7 \end{bmatrix} * \left[\begin{bmatrix} 1 & 4 \\ 3 & 2 \end{bmatrix} - \begin{bmatrix} 2 & 1 \\ 5 & 4 \end{bmatrix} \right]$$

$$= \begin{bmatrix} 1 & 4 \\ 6 & 7 \end{bmatrix} * \begin{bmatrix} -1 & 3 \\ -2 & -2 \end{bmatrix}$$

$$P_4 = \underline{\underline{\begin{bmatrix} -9 & -5 \\ -20 & 4 \end{bmatrix}}}$$

$$P_5 = (a + d) * (e + b)$$

$$= \left[\begin{bmatrix} 4 & 2 \\ 3 & 1 \end{bmatrix} + \begin{bmatrix} 1 & 4 \\ 6 & 7 \end{bmatrix} \right] * \left[\begin{bmatrix} 2 & 1 \\ 5 & 4 \end{bmatrix} + \begin{bmatrix} 0 & 2 \\ 4 & 1 \end{bmatrix} \right]$$

$$= \begin{bmatrix} 5 & 6 \\ 9 & 8 \end{bmatrix} * \begin{bmatrix} 2 & 3 \\ 9 & 5 \end{bmatrix}$$

$$P_5 = \underline{\underline{\begin{bmatrix} 64 & 45 \\ 90 & 67 \end{bmatrix}}}$$

$$P_6 = (b - d) * (g + h)$$

$$= \left[\begin{bmatrix} 0 & 1 \\ 2 & 5 \end{bmatrix} * \begin{bmatrix} 1 & 4 \\ 3 & 7 \end{bmatrix} \right] * \left[\begin{bmatrix} 1 & 4 \\ 3 & 2 \end{bmatrix} + \begin{bmatrix} 0 & 2 \\ 4 & 1 \end{bmatrix} \right]$$

$$= \begin{bmatrix} -1 & 3 \\ -4 & -2 \end{bmatrix} * \begin{bmatrix} 1 & 6 \\ 7 & 3 \end{bmatrix}$$

$$P_6 = \underline{\underline{\begin{bmatrix} -22 & -15 \\ -18 & -30 \end{bmatrix}}}$$

$$P_7 = (a - c) * (e + f)$$

$$= \left[\begin{bmatrix} 4 & 2 \\ 3 & 1 \end{bmatrix} - \begin{bmatrix} 3 & 2 \\ 5 & 2 \end{bmatrix} \right] * \left[\begin{bmatrix} 2 & 1 \\ 5 & 4 \end{bmatrix} + \begin{bmatrix} 3 & 2 \\ 2 & 3 \end{bmatrix} \right]$$

$$P_7 = \begin{bmatrix} 1 & 0 \\ -2 & -1 \end{bmatrix} * \begin{bmatrix} 5 & 3 \\ 7 & 7 \end{bmatrix} = \underline{\underline{\begin{bmatrix} 5 & 3 \\ -17 & -13 \end{bmatrix}}}$$

$$r = P_5 + P_4 - P_2 + P_6$$

$$= \begin{bmatrix} 64 & 45 \\ 90 & 67 \end{bmatrix} + \begin{bmatrix} -9 & -5 \\ -20 & 4 \end{bmatrix} - \begin{bmatrix} 12 & 11 \\ 24 & 16 \end{bmatrix} + \begin{bmatrix} -22 & -15 \\ -18 & -30 \end{bmatrix}$$

$$= \begin{bmatrix} 55 & 40 \\ 70 & 71 \end{bmatrix} - \begin{bmatrix} 12 & 11 \\ 24 & 16 \end{bmatrix} + \begin{bmatrix} -22 & -15 \\ -18 & -30 \end{bmatrix}$$

$$= \begin{bmatrix} 33 & 25 \\ 52 & 47 \end{bmatrix} - \begin{bmatrix} 12 & 11 \\ 24 & 16 \end{bmatrix}$$

$$r = \underline{\underline{\begin{bmatrix} 21 & 14 \\ 28 & 25 \end{bmatrix}}}$$

(Do additions first and then do subtraction)

$$s = P_1 + P_2$$

$$= \begin{bmatrix} 8 & 4 \\ 7 & 2 \end{bmatrix} + \begin{bmatrix} 12 & 11 \\ 24 & 16 \end{bmatrix}$$

$$s = \underline{\underline{\begin{bmatrix} 20 & 15 \\ 31 & 18 \end{bmatrix}}}$$

$$t = P_3 + P_4$$

$$= \begin{bmatrix} 38 & 28 \\ 67 & 47 \end{bmatrix} + \begin{bmatrix} -9 & -5 \\ -20 & 4 \end{bmatrix}$$

$$t = \underline{\underline{\begin{bmatrix} 29 & 23 \\ 47 & 51 \end{bmatrix}}}$$

$$u = P_5 + P_1 - P_3 - P_7$$

$$= \begin{bmatrix} 64 & 45 \\ 90 & 67 \end{bmatrix} + \begin{bmatrix} 8 & 4 \\ 7 & 2 \end{bmatrix} - \begin{bmatrix} 38 & 28 \\ 67 & 47 \end{bmatrix} - \begin{bmatrix} 5 & 3 \\ -17 & -13 \end{bmatrix}$$

$$= \begin{bmatrix} 72 & 49 \\ 97 & 69 \end{bmatrix} - \begin{bmatrix} 38 & 28 \\ 67 & 47 \end{bmatrix} + \begin{bmatrix} 5 & 3 \\ -17 & -13 \end{bmatrix}$$

$$= \begin{bmatrix} 72 & 49 \\ 97 & 69 \end{bmatrix} - \begin{bmatrix} 43 & 31 \\ 50 & 34 \end{bmatrix}$$

Take minus
sign outside
and add

$$u = \begin{bmatrix} 29 & 18 \\ 47 & 35 \end{bmatrix}$$

$$C = \begin{bmatrix} x & s \\ t & u \end{bmatrix} = \begin{bmatrix} 21 & 14 & 20 & 15 \\ 28 & 25 & 31 & 18 \\ 29 & 23 & 29 & 18 \\ 47 & 51 & 47 & 35 \end{bmatrix}$$

Algorithm:- Matrix Multiplication.

Problem: Determine the product of two $n \times n$ matrices

Input: n , a positive integer, A and B - two dimensional array

Output: Two dimensional array C, ie; product of A and B.

```

1 void matrix(int n, int A[][], int B[][])
2 {
3     for i=1 to n do
4     {
5         for j=1 to n do
6         {
7             C[i,j] = 0;
8             for k=1 to n do
9                 C[i,j] = C[i,j] + A[i,k] * B[k,j];
10            }
11        }
12    }

```

MODULE III

Greedy Strategy:-

Greedy method is a straight forward design technique. This method is popular for demanding optimized solutions. The decision of solution is taken based on the information available. The solution is constructed through a sequence of steps.

First we select some solution from input domain then, we check whether the solution is feasible or not. Feasible means it should satisfy the problem's constraints.

From the set of feasible solutions, particular solution that satisfies or nearly satisfies the objective of the function is called optimal solution.

As greedy method works in stages. At each stage only one input is considered at each time. Based on this input it is decided whether particular input gives the optimal solution or not.

ie, These problems have 'n' inputs and require a subset that satisfies some constraints. Any subset that satisfies the constraints are called feasible solution. A feasible solution that either maximizes or minimizes a given objective function is called optimal solution.

The greedy method suggest that one can devise an algorithm that works in stages, considering one input at a time. At each stage a decision is made regarding whether a particular input is an optimal solution. If the inclusion of the input into partially constructed optimal solution will result in an infeasible solution, then this input is not added else it is added. The selection procedure

itself is based on some optimization measure like an objective function. This version of the greedy technique is called the subset paradigm

Control Abstraction for the subset paradigm

```
1. Algorithm Greedy(a, n)
2. // a[1:n] contains the n inputs.
3. {
4.   solution :=  $\emptyset$ ; // Initialize the solution.
5.   for i := 1 to n do
6.   {
7.     x := Select(a);
8.     if Feasible(solution, x) then
9.       solution := Union(solution, x);
10.   }
11.   return solution;
12. }
```

The function Select selects an input from $a[i]$ and removes it. The selected input's value is assigned to x . Feasible is a Boolean-valued function that determines whether x can be included into the solution vector. The function Union combines x with the solution and updates the objective function.

Ordering Paradigm: - The decision is made by considering the inputs in some order. Each decision is made using an optimization criterion that can be computed using decisions already made.

Fractional Knapsack Problem .

Let there are n objects and a knapsack or bag. Every object has a value and a weight. The knapsack has a capacity 'm'. The object i has weight w_i . If a fraction x_i ; $0 \leq x_i \leq 1$, of object i is placed into the knapsack, then a profit of $P_i x_i$ is earned, where $(P_1, P_2, \dots, P_n)$ corresponds to the profit/value of each item. The objective is to fill the knapsack that maximizes the total profit earned. Since the knapsack capacity is m , the total weight of all chosen objects to be atmost m .

The problem is stated as,

$$\text{maximize } \sum_{i=1}^n P_i x_i \quad \text{--- ①}$$

$$\text{subject to } \sum_{i=1}^n w_i x_i \leq m \quad \text{--- ②}$$

$$\text{and } 0 \leq x_i \leq 1 \quad ; \quad 1 \leq i \leq n \quad \text{--- ③}$$

where the profits and weights are positive integers.

A feasible solution is any set $(x_1, \dots, x_n)$ which satisfies ② and ③. An optimal solution is a feasible solution for which ① is maximized.

Fractional Knapsack Problem – Time Complexity

- The for loop will execute maximum n times.
- So the time complexity = $O(n)$

Fractional Knapsack Problem - Algorithm

```

Algorithm GreedyKnapsack(m, n) // m → knapsack capacity
{
    for i= 1 to n do
        x[i] = 0.0;                // x[1:n] → solution vector
    U = m;
    for i=1 to n do
        {
            if w[i] > U then
                break;
            x[i] = 1.0
            U = U - w[i];
        }
    If i ≤ n then
        x[i] = U / w[i];
}

```

U --> current balance
capacity of knapsack

Find the optimal solution for the following fractional Knapsack problem. Given number of items(n)=4, capacity of sack(m) = 60, $W=\{40,10,20,24\}$ and $P=\{280,100,120,120\}$

```

m = 60
n = 4
i  → { 1,   2,   3,   4 }
P  = {280, 100, 120, 120}
W  = {40,  10,  20,  24 }

```

```

m = 60
n = 4
i  → { 1,   2,   3,   4 }
P  = {280, 100, 120, 120}
W  = {40,  10,  20,  24 }
P/W= {7,   10,  6,   5 }

```

$$m = 60$$

$$n = 4$$

$$i \rightarrow \{ 1, \quad 2, \quad 3, \quad 4 \}$$

$$P = \{ 280, \quad 100, \quad 120, \quad 120 \}$$

$$W = \{ 40, \quad 10, \quad 20, \quad 24 \}$$

$$P/W = \{ 7, \quad 10, \quad 6, \quad 5 \}$$

II

I

III

IV

Sort $p[i]/w[i] \geq p[i+1]/w[i+1]$

$$i \rightarrow \{ 2, \quad 1, \quad 3, \quad 4 \}$$

$$P = \{ 100, \quad 280, \quad 120, \quad 120 \}$$

$$W = \{ 10, \quad 40, \quad 20, \quad 24 \}$$

i	Pi	Wi	Xi	U = U - Wi
2	100	10		
1	280	40		
3	120	20		
4	120	24		

$$U = m = 60$$

fraction of
weight of
object to
be placed

knapsack balance
capacity

i	Pi	Wi	Xi	U = U - Wi
2	100	10	0	
1	280	40	0	
3	120	20	0	
4	120	24	0	

$$U = m = 60$$

i	Pi	Wi	Xi	U = U-Wi
2	100	10	1	
1	280	40	0	
3	120	20	0	
4	120	24	0	

$$U = m = 60$$

i	Pi	Wi	Xi	U = U-Wi
2	100	10	1	50
1	280	40	0	
3	120	20	0	
4	120	24	0	

$$U = m = 60$$

i	Pi	Wi	Xi	U = U-Wi
2	100	10	1	50
1	280	40	1	10
3	120	20	0	
4	120	24	0	

i	Pi	Wi	Xi	U = U-Wi
2	100	10	1	50
1	280	40	1	10
3	120	20	1/2	
4	120	24	0	

i	Pi	Wi	Xi	U = U-Wi
2	100	10	1	50
1	280	40	1	10
3	120	20	1/2	0
4	120	24	0	

$$\text{Total Profit} = \sum P_i * X_i$$

$$= 100 \times 1 + 280 \times 1 + 120 \times 1/2 = \mathbf{440}$$

i	Pi	Wi	Xi	U = U-Wi
2	100	10	1	50
1	280	40	1	10
3	120	20	1/2	0
4	120	24	0	

$$\text{Total Profit} = \sum P_i * X_i$$

$$= 100 \times 1 + 280 \times 1 + 120 \times 1/2 = \mathbf{440}$$

Solution vector **$X = \{1, 1, 1/2, 0\}$**

sort solution vector as per i (1 to 4)

Find an optimal solution to the fractional knapsack problem for an instance with number of items 7, Capacity of the sack =15, $(p_1, p_2, \dots, p_7) = (10, 5, 15, 7, 6, 18, 3)$ and $(w_1, w_2, \dots, w_7) = (2, 3, 5, 7, 1, 4, 1)$.

$$m = 15$$

$$n = 7$$

$$i \rightarrow \{1, 2, 3, 4, 5, 6, 7\}$$

$$P = \{10, 5, 15, 7, 6, 18, 3\}$$

$$W = \{2, 3, 5, 7, 1, 4, 1\}$$

$$m = 15$$

$$n = 7$$

$$i \rightarrow \{1, 2, 3, 4, 5, 6, 7\}$$

$$P = \{10, 5, 15, 7, 6, 18, 3\}$$

$$W = \{2, 3, 5, 7, 1, 4, 1\}$$

$$P/W = \{5, 1.67, 3, 1, 6, 4.5, 3\}$$

$$m = 15$$

$$n = 7$$

$$i \rightarrow \{1, 2, 3, 4, 5, 6, 7\}$$

$$P = \{10, 5, 15, 7, 6, 18, 3\}$$

$$W = \{2, 3, 5, 7, 1, 4, 1\}$$

$$P/W = \{5, 1.67, 3, 1, 6, 4.5, 3\}$$

Sort $p[i]/w[i] \geq p[i+1]/w[i+1]$

$$i \rightarrow \{5, 1, 6, 3, 7, 2, 4\}$$

$$P = \{6, 10, 18, 15, 3, 5, 7\}$$

$$W = \{1, 2, 4, 5, 1, 3, 7\}$$

i	Pi	Wi	Xi	U = U-Wi
5	6	1	0	
1	10	2	0	
6	18	4	0	
3	15	5	0	
7	3	1	0	
2	5	3	0	
4	7	7	0	

$$U = m = 15$$

i	Pi	Wi	Xi	U = U-Wi
5	6	1	1	14
1	10	2	1	12
6	18	4	1	8
3	15	5	1	3
7	3	1	1	2
2	5	3	2/3	0
4	7	7	0	

$$\text{Total Profit} = \sum P_i * X_i$$

$$= 6 \times 1 + 10 \times 1 + 18 \times 1 + 15 \times 1 + 3 \times 1 + 5 \times \frac{2}{3} = \mathbf{55.33}$$

$$\text{Solution vector } \mathbf{X} = \{1, \frac{2}{3}, 1, 0, 1, 1, 1\}$$

sort solution vector as per i (1 to 7)

Qn) Find the optimal solution for knapsack problem where $m=25$, $n=3$, $P=(25,24,17)$, $w=(16,14,9)$

Ans) 44.125

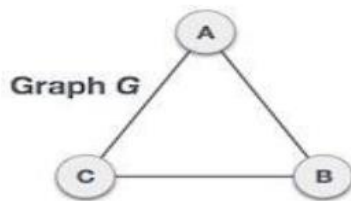
Spanning Trees

A spanning tree is a subset of undirected connected Graph $G=(V,E)$, which has all the vertices covered with minimum possible number of edges.

It does not have cycles and cannot be disconnected.

To construct a spanning tree,

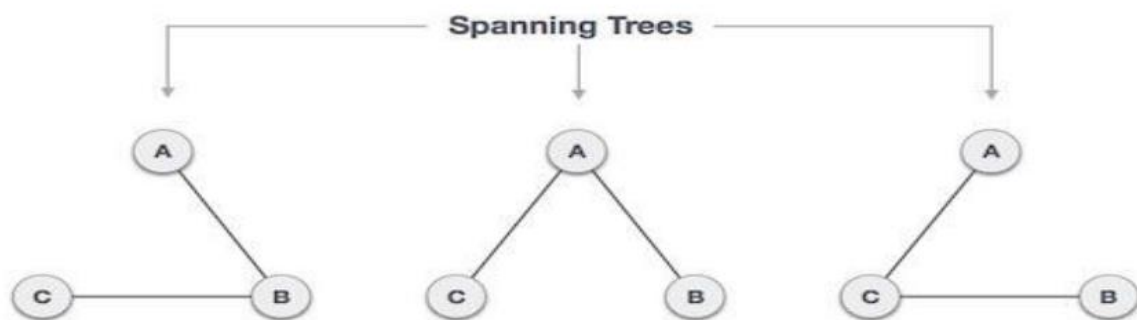
- From a complete graph - removing **$\max(e-n+1)$ edges**,
where e = no. of edges, n = no. of vertices
- Spanning tree will have **$(n-1)$ edges**.



Here, $n=3$, $e=3$

Therefore, number of edges in spanning tree = $(n-1) = 2$ or

(we remove $(e-n+1)$ edges, i.e. $3-3+1 = 1$ edges)



Properties of Spanning Tree

- A connected graph G can have more than one spanning tree.
- All possible spanning trees of graph G , have the same number of edges and vertices.
- The spanning tree does not have any cycle (loops)
- Removing one edge from the spanning tree will make the graph disconnected, i.e. the spanning tree is minimally connected
- Adding one edge to the spanning tree will create a circuit or loop, i.e. the spanning tree is maximally acyclic
- Spanning tree has $n-1$ edges, where n is the number of nodes.

Application of Spanning Tree

- Civil Network Planning
- Computer Network Routing Protocol
- Cluster Analysis
- Handwriting Recognition
- Image Segmentation

Maximum number of Spanning Trees of a graph with n nodes

- Complete Graph: n^{n-2}

Other Graphs

1. Create Adjacency Matrix for the given graph.
2. Replace all the diagonal elements with the degree of nodes.
3. Replace all non-diagonal 1's with -1.
4. Total number of spanning tree for that graph = Co-factor for any element in that matrix.

Qtn) Write the total number of spanning trees possible for a complete graph with 6 vertices

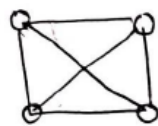
- Total number of spanning trees possible for a complete graph with n vertices = n^{n-2}
- Total number of spanning trees possible for a complete graph with 6 vertices = $6^{6-2} = 6^4 = 1296$

Minimum Spanning Tree (MST)

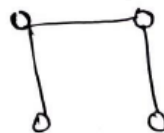
- In a weighted graph, a minimum spanning tree is a spanning tree that has minimum weight than all other spanning trees of the same graph.
- In real-world situations, this weight can be measured as distance, congestion, traffic load or any arbitrary value denoted to the edges.

Minimum Cost Spanning Trees.

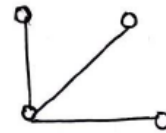
Let $G=(V,E)$ be an undirected connected graph. A subgraph $t=(V,E')$ of G is a spanning tree of G iff t is a tree



(a)



(b)

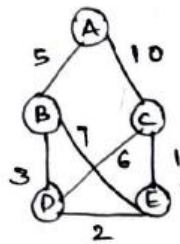


(c)

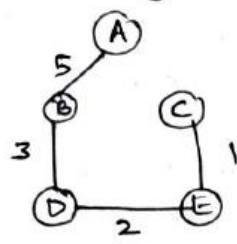
(a) shows a complete graph on 4 nodes
(b) & (c) are the two spanning trees of (a).

A spanning tree is a subgraph of a graph which contains all the vertices, but not all the edges in the tree. If the sum of the weights of all

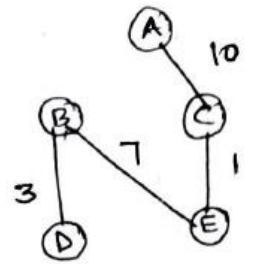
edges in the tree is minimum then it is called minimum spanning tree. Not every spanning tree has minimum weight.



Graph G



$Wt(T_1) = 11$



$Wt(T_2) = 16$

So, T_1 is the minimum spanning tree. Graph G is a weighted connected graph and T_1, T_2 are the spanning trees.

Application of Spanning Tree

- Civil Network Planning
- Computer Network Routing Protocol
- Cluster Analysis
- Handwriting Recognition
- Image Segmentation

Qn) Let (u,v) be a minimum-weight edge in a graph G. Show that (u,v) belongs to some minimum spanning tree of G.

- Suppose that T is a Minimum Spanning Tree, which does not include the smallest edge, E.
- Add E to T. Now a circle C is formed.
- This graph will remain connected if an edge is removed from the circle C.
- So remove an edge E'' (except E) from C which also belongs to T
- This operation would result a new spanning tree whose weight is $\leq$ weight of T.
- We have a contradiction. Hence, proved.

Qn) Let G be a weighted undirected graph with distinct positive edge weights. If every edge weight is increased by same value, will the minimum cost spanning tree change. Justify your answer

- The Minimum Spanning Tree doesn't change.
- In Kruskal's algorithm, we will sort the edges first.
- If we increase all weights, then order of edges won't change.
- So, MST does not change.

Minimum Spanning-Tree Algorithms

- Prim's Algorithm
- **Kruskal's Algorithm**

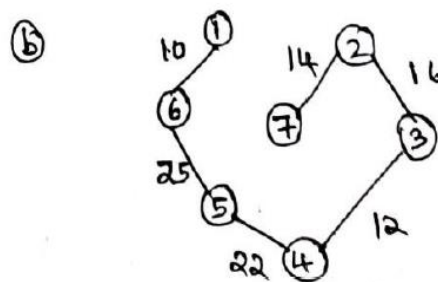
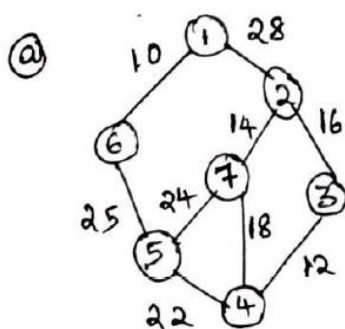
PRIM'S ALGORITHM

- It is a greedy algorithm that finds MST for a **connected weighted undirected graph**.
- It finds the subset of edges that form a tree that includes every vertex where total weight of all the edges in a tree is **minimized**.

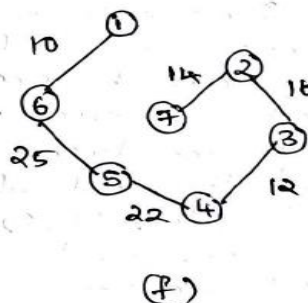
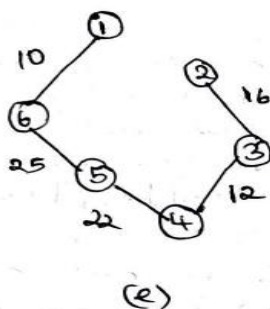
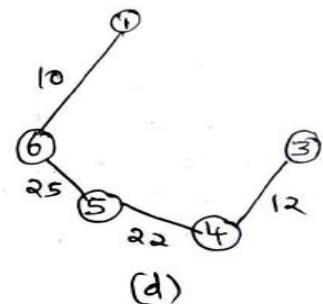
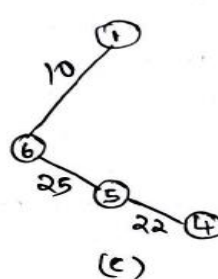
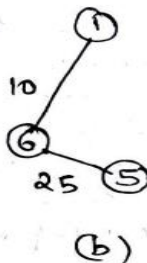
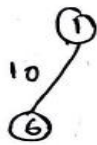
STEPS

1. Initialise the MST with a vertex chosen at random.
2. Find all the edges that connect the tree to new vertices. Find the minimum and add it to the tree unless it forms a cycle.
3. Keep repeating step 2 until we get a MST.

A graph (a) and its minimum cost spanning tree (b).



Stages in Prim's Algorithm.



Minimum cost = 99

Complexity of Prim's algorithm- $O(n^2)$

Kruskal's Algorithm:-

A greedy method to generate minimum cost spanning tree. First, the edge with minimum cost is added to the tree, then next edge with minimum cost is joined with this. and the algorithm proceeds. It is not necessary that the next edge is an adjacent edge. Cycles should not be formed.

In this algorithm, the edges of the graph are considered in increasing order of cost.

Steps:-

- List all edges of the graph G in increasing order of weights
- Select the smallest edge from the list and add to the spanning tree if the inclusion of the edge doesnot make a cycle.
- If the selected edge with smallest weight forms a cycle, remove it from the list.
- Repeat steps 2 and 3 until the tree contains $(n-1)$ edges or the list is empty.
- If the tree contains less than $(n-1)$ edges and the list is empty, print 'there is no spanning tree possible', otherwise return spanning tree.

Algorithm Kruskals(E , cost, n , t)

```
{
    Construct a heap out of edge costs using Heapify();
    for  $i=1$  to  $n$  do
        parent[ $i$ ] = -1;
     $i=0$ ;
    mincost = 0.0;
    while ( $i < n-1$ ) and (heap not empty) do
    {
        Delete a minimum cost edge ( $u, v$ ) from the heap and reheapify using Adjust();
         $j = \text{Find}(u)$ ;
         $k = \text{Find}(v)$ ;
        if  $j \neq k$  then
```



```

    {
        i = i+1;
        t[i, 1] = u;    t[i, 2] = v;
        mincost = mincost + cost[u, v];
        Union(j, k);
    }
}
if i ≠ n-1 then
    Write ("No Spanning Tree");
else
    return mincost;
}

```

E is the set of edges and n is the number of vertices in G.

cost[u,v] is the cost of edge (u, v).

t is the set of edges in the minimum cost spanning tree.

The final cost is returned

- Heapify() is used to construct a minheap based on the edge cost of G.
- Adjust() is used to reconstruct a minheap if there is a deletion occurs.
- Initially all vertices are belongs to different sets. Find() returns the set number of that particular vertex. j and k are the set number of vertex u and v respectively.
- If j=k means vertex u and v are belongs to the same set. Inclusion of (u, v) should definitely form a cycle. So discard it.
- If j≠k means vertex u and v are belongs to different set. Inclusion of (u, v) will not form a cycle. So add it to the minimum spanning tree edge list.
- Finally there are n-1 edges, then retrun it. Otherwise there is no spanning tree.

Complexity

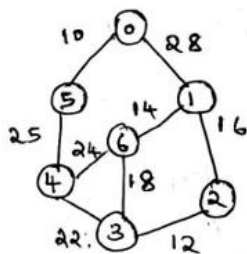
- The edges are maintained as a minheap, then the next edge to consider can be obtained in **$O(\log |E|)$** time.
- Construction of heap itself takes **$O(|E|)$** time.
- Overall complexity of Kruskal's algorithm is **$O(|E| \log|E|)$** .

Difference between Prim's & Kruskal's Algorithm

Feature	Prim's Algorithm	Kruskal's Algorithm
Approach	Vertex-based, grows the MST one vertex at a time	Edge-based, adds edges in increasing order of weight
Data Structure	Priority queue (min-heap)	Union-Find data structure
Graph Representation	Adjacency matrix or adjacency list	Edge list

Initialization	Starts from an arbitrary vertex	Starts with all vertices as separate trees (forest)
Edge Selection	Chooses the minimum weight edge from the connected vertices	Chooses the minimum weight edge from all edges
Cycle Management	Not explicitly managed; grows connected component	Uses Union-Find to avoid cycles
Complexity	$O(V^2)$ for adjacency matrix, $O((E + V) \log V)$ with a priority queue	$O(E \log E)$ or $O(E \log V)$, due to edge sorting
Suitable for	Dense graphs	Sparse graphs
Implementation Complexity	Relatively simpler in dense graphs	More complex due to cycle management
Parallelism	Difficult to parallelize	Easier to parallelize edge sorting and union operations
Memory Usage	More memory for priority queue	Less memory if edges can be sorted externally
Example Use Cases	Network design, clustering with dense connections	Road networks, telecommunications with sparse connections
Starting Point	Requires a starting vertex	No specific starting point, operates on global edges
Optimal for	Dense graphs where adjacency list is used	Sparse graphs where edge list is efficient

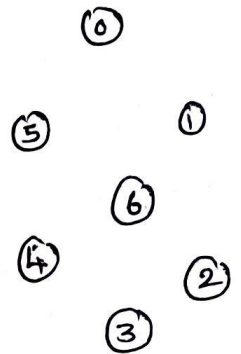
1) solve using Kruskal's Algorithm?



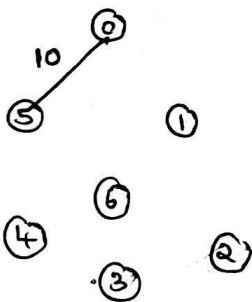
2) Initially, list all the edges of Graph G in increasing order of weights.

Edge	Weight	Result
(0,5)	10	Added.
(2,3)	12	Added
(1,6)	14	Added
(1,2)	16	Added.
(3,6)	18	Discarded.
(3,4)	22	Added.
(4,6)	24	Discarded
(4,5)	25	Added.
(0,1)	28	Discarded.

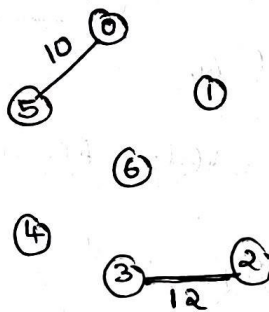
Initially,



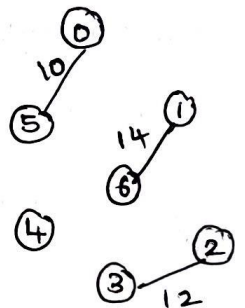
(a) Insert (0,5)



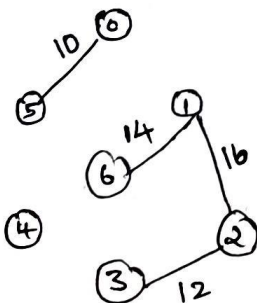
(b) Insert (2,3)



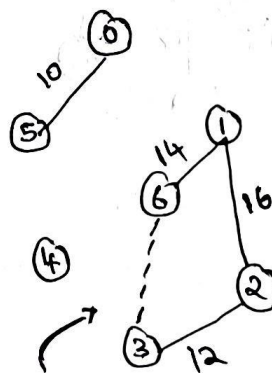
(c) Insert (1,6)



(d) Insert (1,2)

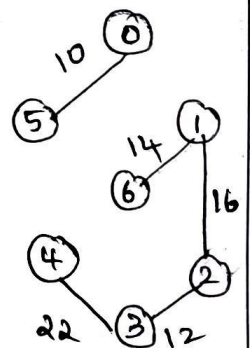


(e) Insert (3,6)

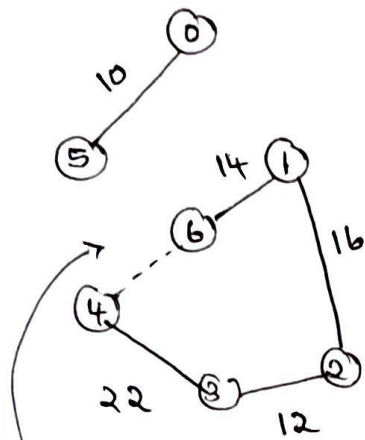


Leads to Cycle
∴ Discard

(f) Insert (3,4)

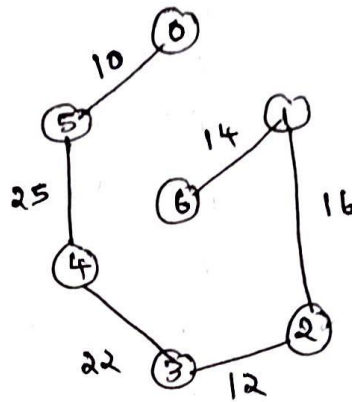


(g) Insert (4,6)

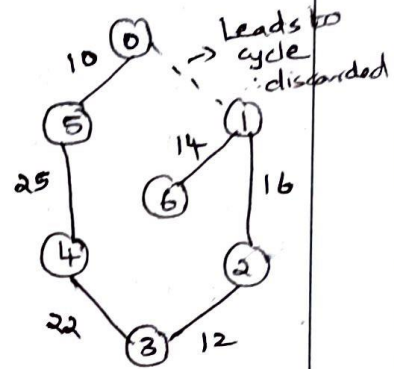


heads to cycle
 $\therefore$ discarded.

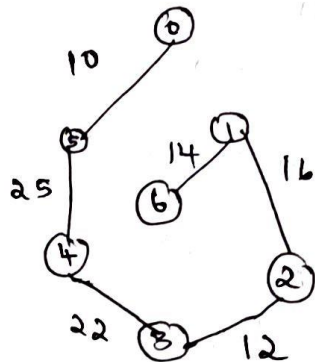
(h) Insert (4,5)



(i) Insert (0,1)



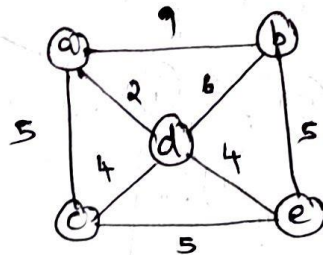
So, Minimum cost spanning tree is



$$\begin{aligned} \text{Minimum Cost} &= 10 + 25 + 22 + \\ &\quad 12 + 16 + 14 \\ &= \underline{\underline{99}} \end{aligned}$$

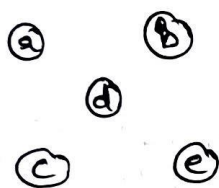
$$\boxed{\text{Minimum Cost} = \underline{\underline{99}}}$$

2) Solve, using Kruskal's algorithm and find minimum cost spanning Tree.

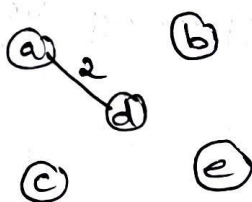


Edge	Weight	Result
(a,d)	2	Added
(c,d)	4	Added
(e,d)	4	Added
(a,c)	5	Discarded
(b,e)	5	Added
(e,e)	5	Discarded
(b,d)	6	Discarded
(a,b)	9	Discarded

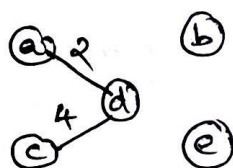
a) Initially



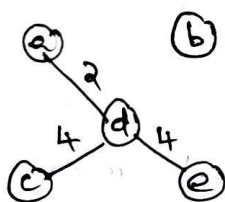
b) Insert (a,d)



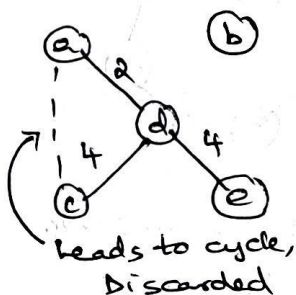
c) Insert (c,d)



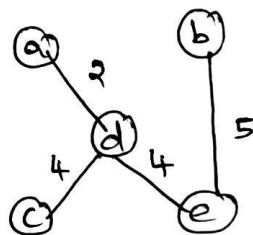
d) Insert (e,d)



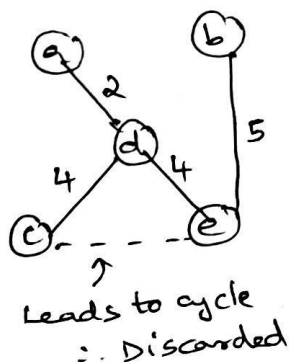
e) Insert (a,c)



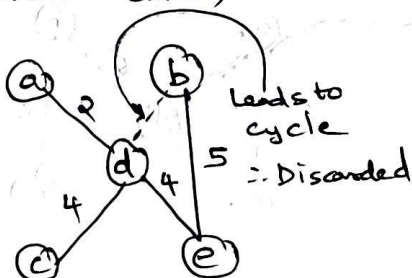
f) Insert (b,e)



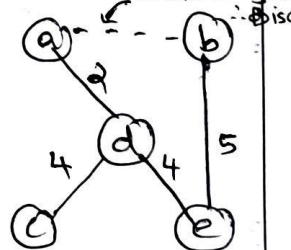
g) Insert (c,e)



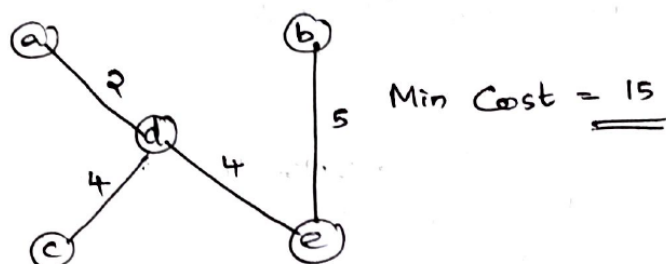
h) Insert (b,d)



i) Insert (a,b)

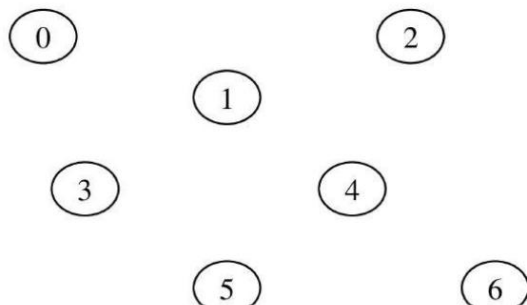
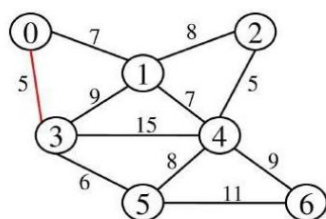
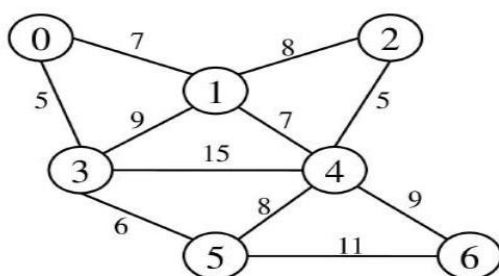


∴ Minimum Cost spanning Tree,



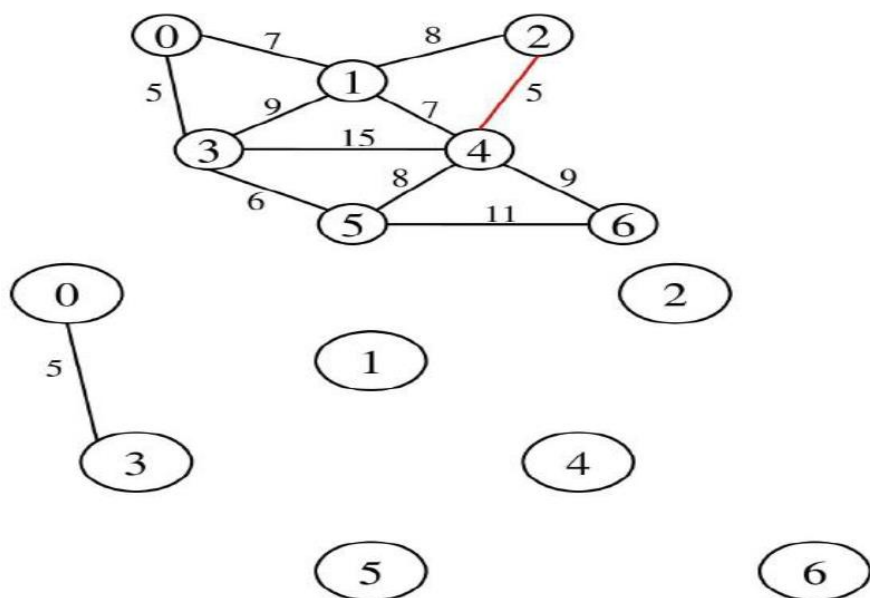
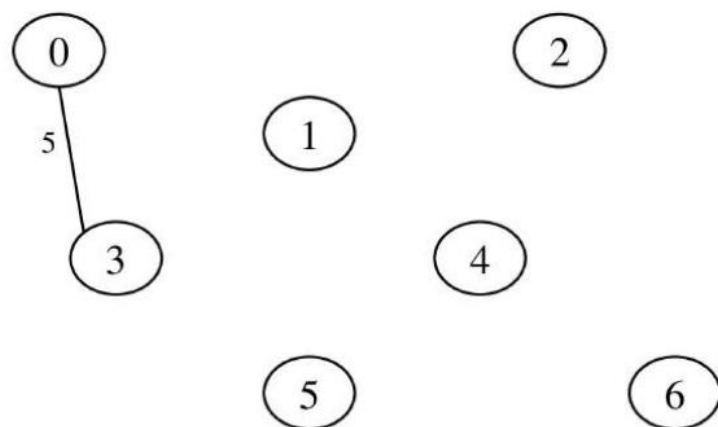
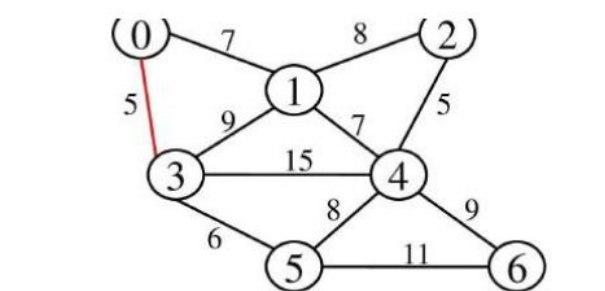
Kruskal's Algorithm - Example

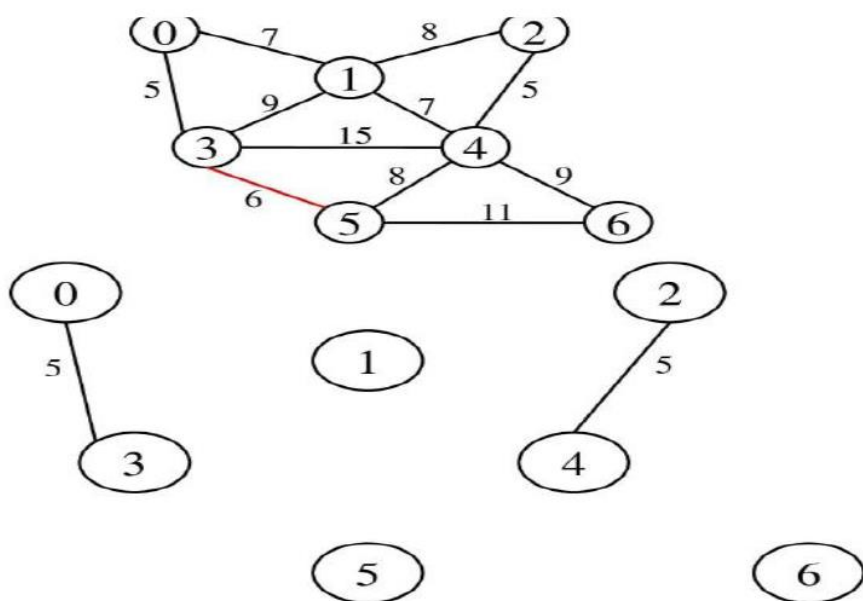
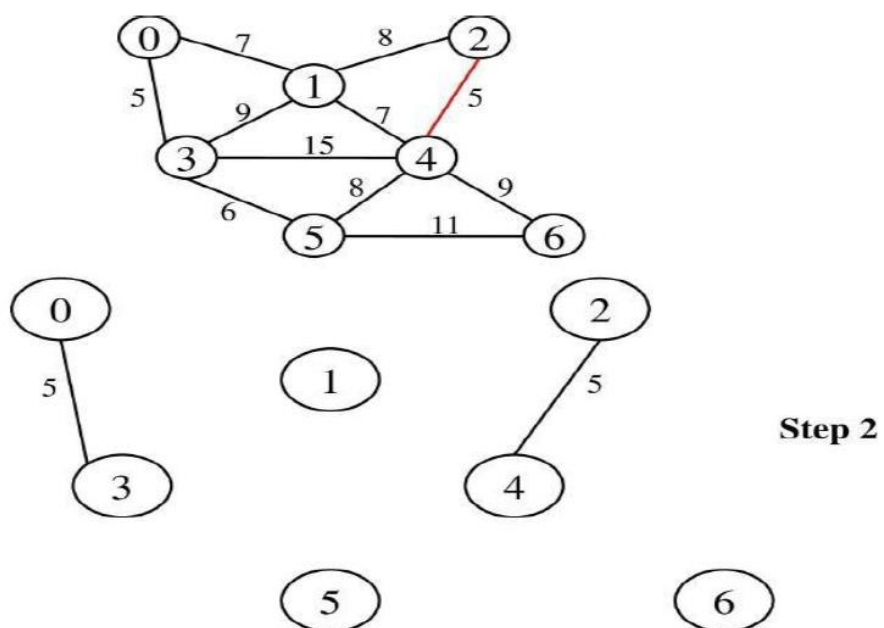
Compute the minimum spanning tree and its cost for the following graph using Kruskal's algorithm

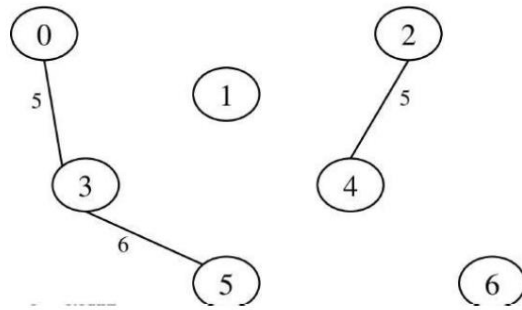
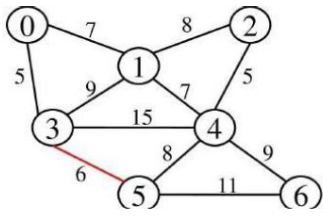


EDGE WEIGHT RESULT

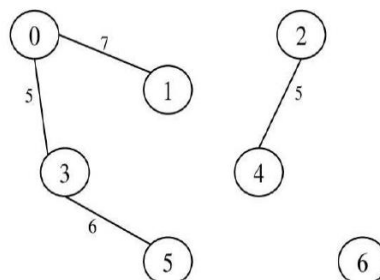
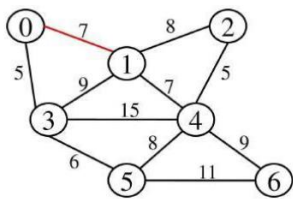
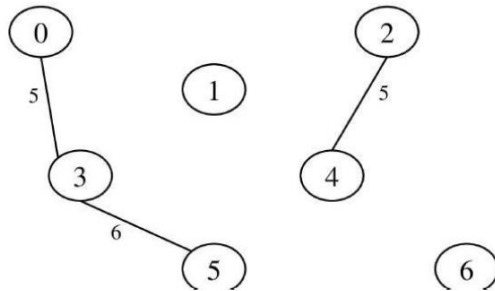
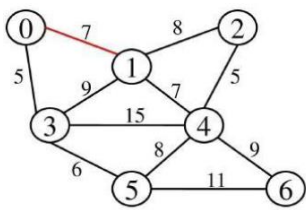
(0,3)	5	added
(2,4)	5	added
(3,5)	6	added
(0,1)	7	added
(1,4)	7	added
(1,2)	8	discarded
(4,5)	8	discarded
(1,3)	9	discarded
(4,6)	9	added
(5,6)	11	discarded
(3,4)	15	discarded



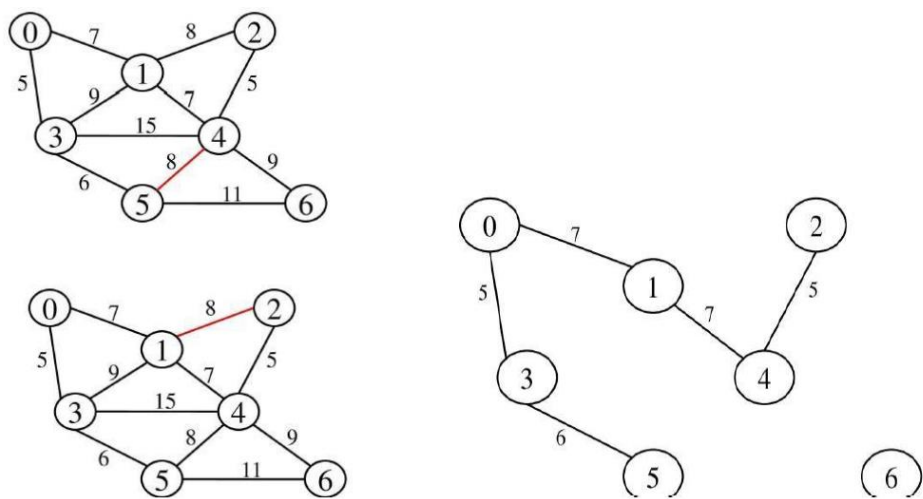
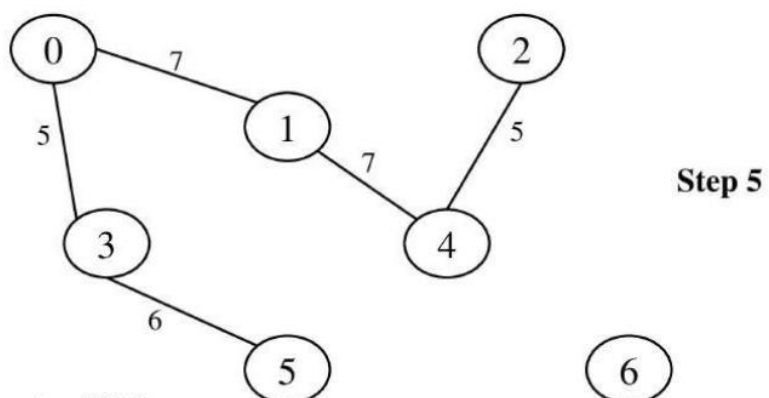
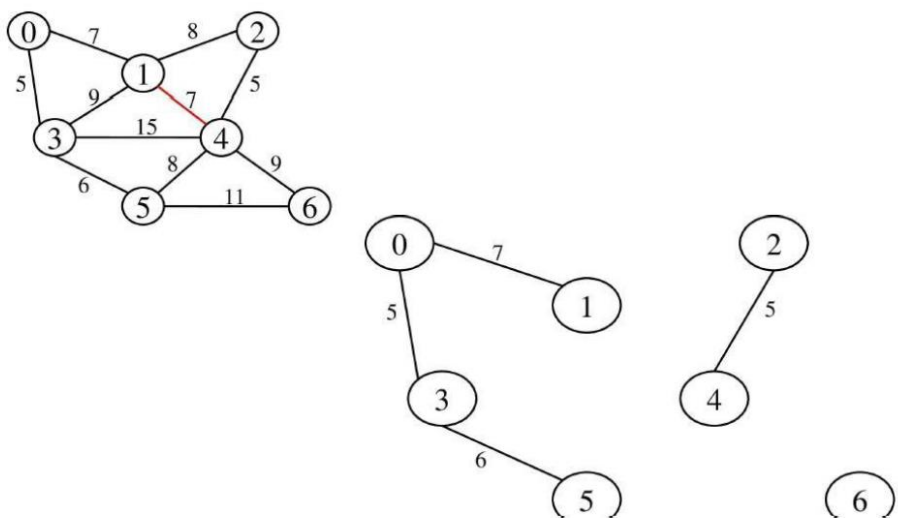


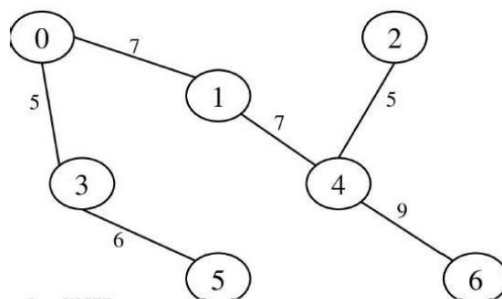
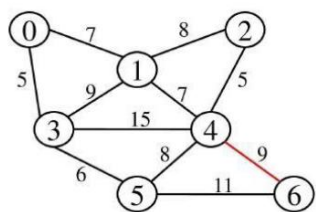


Step 3

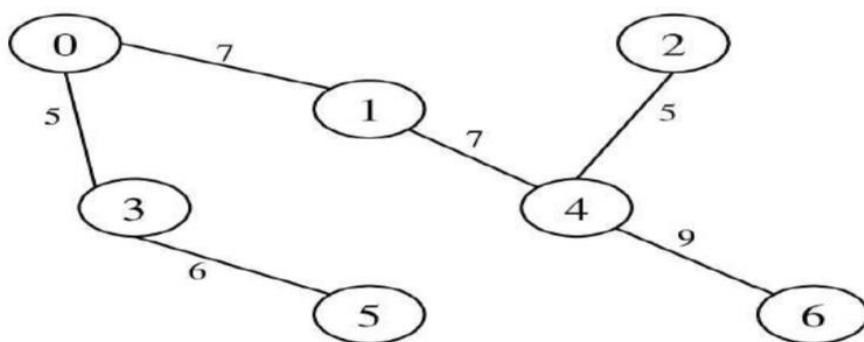


Step 4





Step 6



Minimum Cost Spanning Tree

Cost = 39

Shortest Path Algorithms

The shortest path problem is the problem of finding a path between two vertices in a graph such that the sum of the weights of its constituent edges is minimized.

Different shortest path problems are:

(1) Single Source Shortest Path Problem: To find shortest paths from a source vertex to all other vertices in the directed graph

(2) Single Destination Shortest Path Problem: To find shortest paths from all vertices in the directed graph to a single destination vertex v

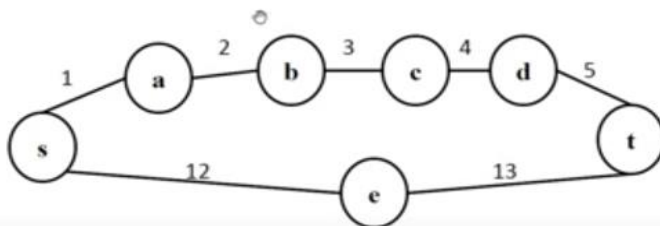
(3) All Pairs Shortest Path Problem: To find shortest paths between every pair of vertices in the graph

Qn 1) Let G be a weighted undirected graph with distinct positive edge weights. If every edge weight is increased by same value, will the shortest path between any pair of vertices change. Justify your answer

The shortest path may change.

- There may be different paths from s to t .
- Let shortest path(Path-1) be of cost 15 and has 5 edges.
- Let there be another path(Path-2) of cost 25 and has 2 edges.
- All edge costs are increased by 10.
- Path-1 cost is increased by 5×10 and becomes $15 + 50 = 65$.
- Path-2 cost is increased by 2×10 and becomes $25 + 20 = 45$
- Now Path-1 cost $>$ Path-2 cost
- So the shortest path may change.

Proof: Let G be a weighted undirected graph with distinct positive edge weights. If every edge weight is increased by same value, will the shortest path between any pair of vertices change. Justify your answer



From s to t , there are two paths:

Path 1: $(s-a-b-c-d-t)$ and

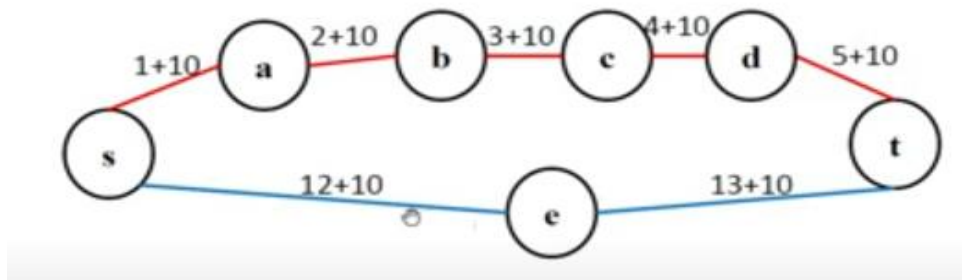
Path 2: $(s-e-t)$.

Cost of $s-a-b-c-d-t$ is 15 and

Cost of $s-e-t = 25$

Therefore, **Shortest path is Path 1: $s-a-b-c-d-t$ and shortest path cost is 15**

Now, all edge costs are increased by 10.

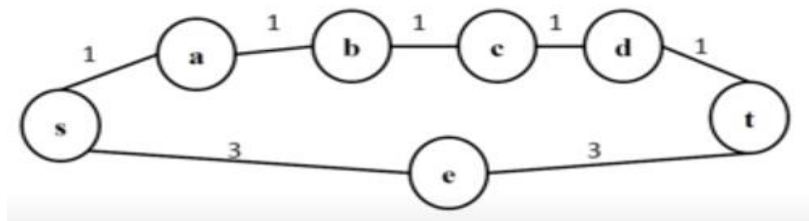


- Path-1 cost is increased by 5×10 and (s-a-b-c-d-t) becomes $15 + 50 = 65$.
- Path-2 cost is increased by 2×10 and (s-e-t) becomes $25 + 20 = 45$

Now path 2 is shortest.

Therefore, if every edge weight is increased by same value, the shortest path between any pair of vertices may change.

Qn 2) In a weighted graph, assume that the shortest path from a source 's' to a destination 't' is correctly calculated using a shortest path algorithm. Is the following statement true? If we increase weight of every edge by 1, the shortest path always remains same. Justify your answer with proper example.



From s to t, there are two paths:

Path 1: (s-a-b-c-d-t) and

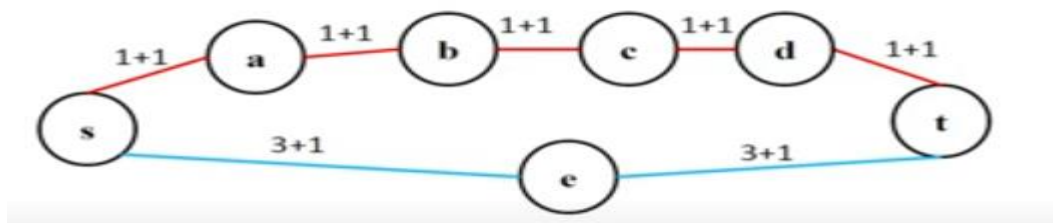
Path 2: (s-e-t).

Cost of s-a-b-c-d-t is 5 and

Cost of s-e-t = 6

Therefore, **Shortest path is Path 1: (s-a-b-c-d-t) and shortest path cost is 5**

Now, all edge costs are increased by 1



- Path-1 (s-a-b-c-d-t) cost is increased by 5×1 and becomes 10
- Path-2 (s-e-t) cost is increased by 2×1 and becomes 8

Now path 2 is shortest

Therefore, if every edge weight is increased by 1, the shortest path between any pair of vertices may change.

Single Source Shortest Path Problem

- Given a connected weighted graph $G=(V,E)$, find the shortest path from a given source vertex s to every other vertices ($V-\{s\}$) in the graph.
- The weight of any path($w(p)$) is the sum of the weights of its constituted edges.
- The weight of the shortest path from u to $v = \min\{w(p): p \text{ is a path from } u \text{ to } v\}$

Single Source Shortest Path Algorithms are:

- Dijkstra's Algorithm
- Bellman Ford Algorithm

Dijkstra's Algorithm

Given a graph and a source vertex S in graph, find shortest paths from S to all vertices in the given graph.

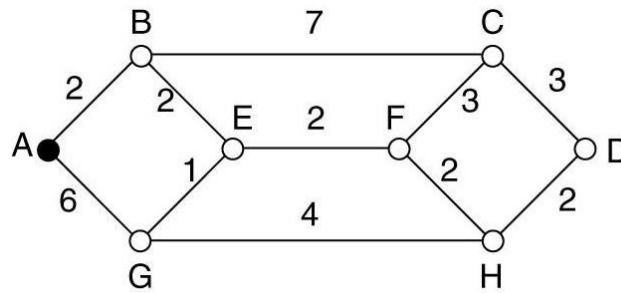
Algorithm Dijkstra(G, W, S)

1. For each vertex v in G
 $\text{distance}[v] = \text{infinity}$
 $\text{previous}[v] = \text{Null}$
2. $\text{distance}[S] = 0$
3. $Q = \text{set of vertices of graph } G$
4. While Q is not empty
 $u = \text{vertex in } Q \text{ with minimum distance}$
 remove u from Q
 for each neighbor v of u which is still in Q
 $\text{alt} = \text{distance}[u] + W(u,v)$
 if $\text{alt} < \text{distance}[v]$
 $\text{distance}[v] = \text{alt}$
 $\text{previous}[v] = u$
5. Return $\text{distance}[], \text{previous}[]$

Complexity

- The complexity mainly depends on the implementation of Q
- The simplest version of Dijkstra's algorithm stores the vertex set Q as an ordinary linked list or array, and extract-minimum is simply a linear search through all vertices in Q . In this case, the running time is $O(E + V^2) = O(V^2)$
- Graph represented using adjacency list can be reduced to $O(E \log V)$ with the help of binary heap.

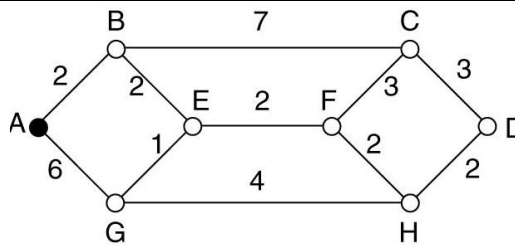
- Find the shortest path from node A to all nodes using Dijkstra's shortest path algorithm



Ans)

- Start from node A. Check the adjacent nodes of A ie; B & G.

Visited	A	B	C	D	E	F	G	H
Initially	0	∞	∞	∞	∞	∞	∞	∞

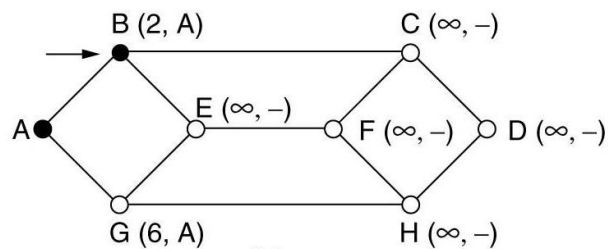


(a)

- Update the labels of B & G with distance & path.

Visited	A	B	C	D	E	F	G	H
{A}	0	2	∞	∞	∞	∞	6	∞

Now move to node B (as the distance of B is smaller when compared to the all other distances). Mark an arrow at B

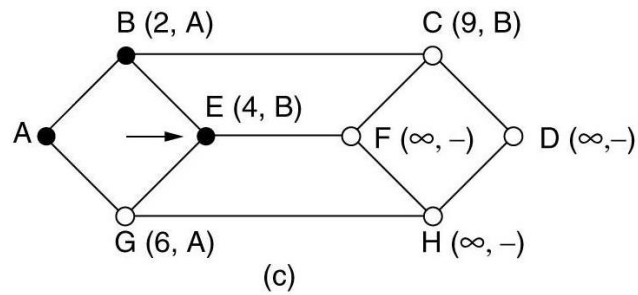


(b)

- Check the adjacent nodes of B ie; C & E and update its labels

Visited	A	B	C	D	E	F	G	H
{A}	0	2	∞	∞	∞	∞	6	∞
{A,B}	0	2	9	∞	4	∞	6	∞

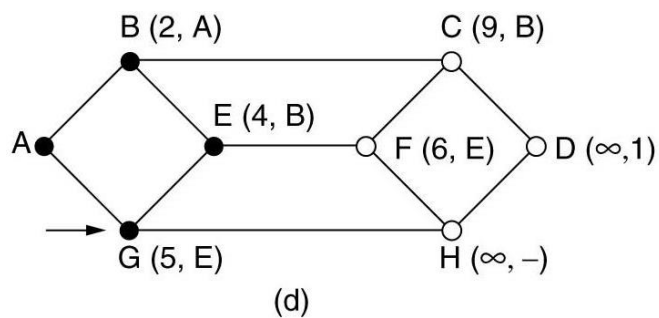
Now move to node E (as the distance of E is smaller). Mark an arrow at E



d) Check the adjacent nodes of E ie; F & G and update its labels

Visited	A	B	C	D	E	F	G	H
{A}	0	2	∞	∞	∞	∞	6	∞
{A,B}	0	2	9	∞	4	∞	6	∞
{A,B,E}	0	2	9	∞	4	6	5	∞

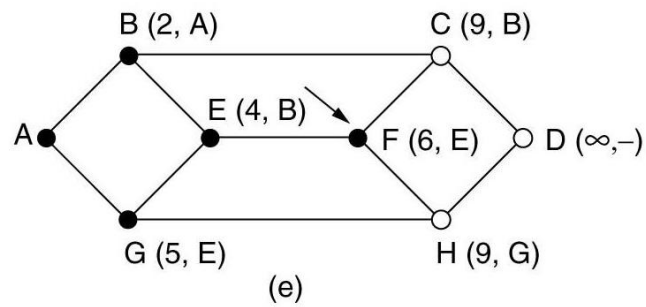
Now move to node G (as the distance of G is smaller). Mark an arrow at G



e) Check the adjacent nodes of G ie; H & E. But E is already visited, so update label of H.

Visited	A	B	C	D	E	F	G	H
{A}	0	2	∞	∞	∞	∞	6	∞
{A,B}	0	2	9	∞	4	∞	6	∞
{A,B,E}	0	2	9	∞	4	6	5	∞
{A,B,E,G}	0	2	9	∞	4	6	5	9

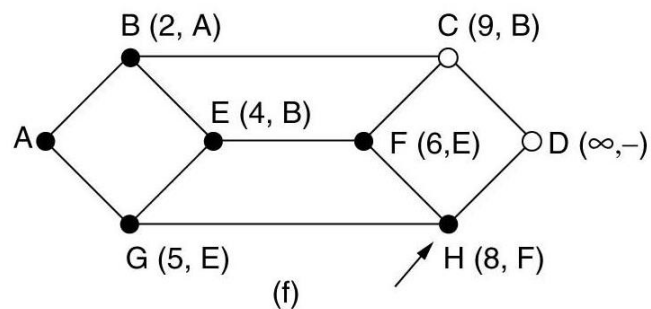
Now move to node F (as the distance of F is smaller). Mark an arrow at F



f) Check the adjacent nodes of F ie; C & H and update its labels

Visited	A	B	C	D	E	F	G	H
{A}	0	2	∞	∞	∞	∞	6	∞
{A,B}	0	2	9	∞	4	∞	6	∞
{A,B,E}	0	2	9	∞	4	6	5	∞
{A,B,E,G}	0	2	9	∞	4	6	5	9
{A,B,E,G,F}	0	2	9	∞	4	6	5	8

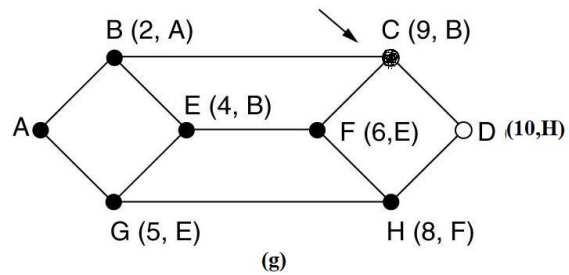
Now move to node H (as the distance of H is smaller). Mark an arrow at H



g) Check the adjacent nodes of H ie; G, D & F. But G, F are already visited, so update label of D.

Visited	A	B	C	D	E	F	G	H
{A}	0	2	∞	∞	∞	∞	6	∞
{A,B}	0	2	9	∞	4	∞	6	∞
{A,B,E}	0	2	9	∞	4	6	5	∞
{A,B,E,G}	0	2	9	∞	4	6	5	9
{A,B,E,G,F}	0	2	9	∞	4	6	5	8
{A,B,E,G,F,H}	0	2	9	10	4	6	5	8

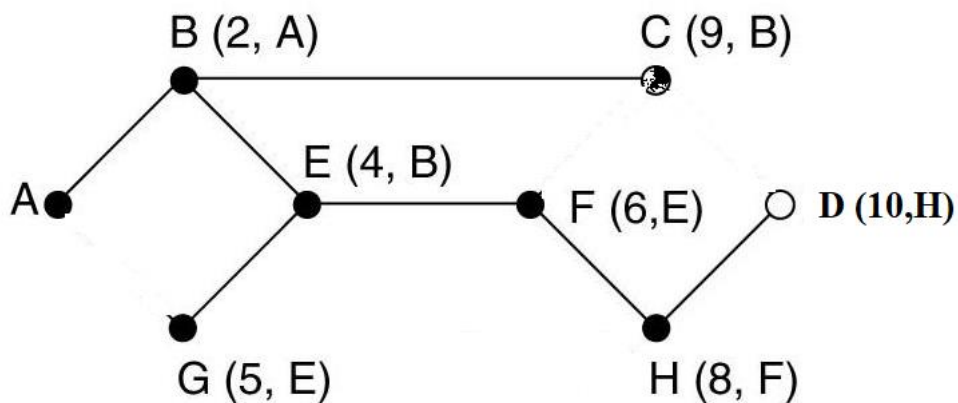
Now move to node C (as the distance of C is smaller). Mark an arrow at C



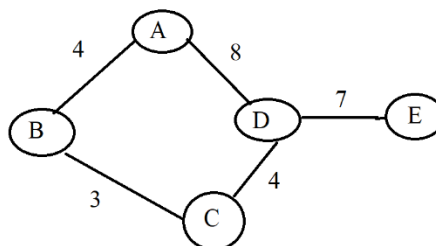
- h) Check the adjacent nodes of C ie; D & F. But F is already visited, so update label of D (no change as 10 is shortest).

Visited	A	B	C	D	E	F	G	H
{A}	0	2	∞	∞	∞	∞	6	∞
{A,B}	0	2	9	∞	4	∞	6	∞
{A,B,E}	0	2	9	∞	4	6	5	∞
{A,B,E,G}	0	2	9	∞	4	6	5	9
{A,B,E,G,F}	0	2	9	∞	4	6	5	8
{A,B,E,G,F,H}	0	2	9	10	4	6	5	8
{A,B,E,G,F,H,C}	0	2	9	10	4	6	5	8

Thus all nodes are visited from A & the shortest route is

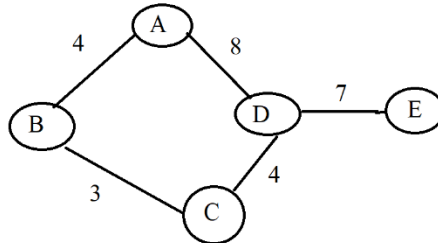


2. Find the shortest path from node A to all nodes using Dijkstra's shortest path algorithm



Ans) Start from node A.

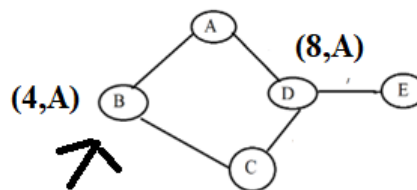
Visited	A	B	C	D	E
Initially	0	∞	∞	∞	∞



Check the adjacent nodes of A ie; B & D. Update the labels of B & D with distance & path.

Visited	A	B	C	D	E
{A}	0	4	∞	8	∞

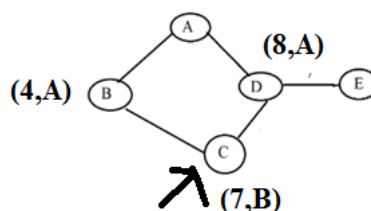
Now move to node B (as the distance of B is smaller when compared to the all other distances). Mark an arrow at B.



Check the adjacent nodes of B ie; C & A. A is the root node so update the labels of C only with distance & path.

Visited	A	B	C	D	E
{A}	0	4	∞	8	∞
{A,B}	0	4	7	8	∞

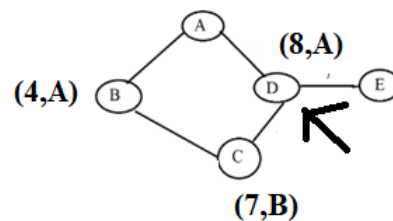
Now move to node C (as the distance of C is smaller when compared to the all other distances). Mark an arrow at C.



Check the adjacent nodes of C ie; B & D. B is visited node, so update the labels of D only with distance & path. But D's previous cost 8 which is smaller to present cost 11 from C, so no need to update cost of D.

Visited	A	B	C	D	E
{A}	0	4	∞	8	∞
{A,B}	0	4	7	8	∞
{A,B,C}	0	4	7	8	∞

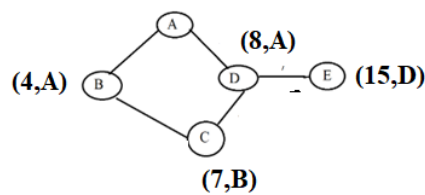
Now move to node D (as the distance of C is smaller when compared to the all other distances).
Mark an arrow at D.



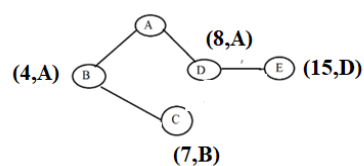
Check the adjacent nodes of D ie; E, A, C.

A & C are visited nodes, so update the labels of E only with distance & path.

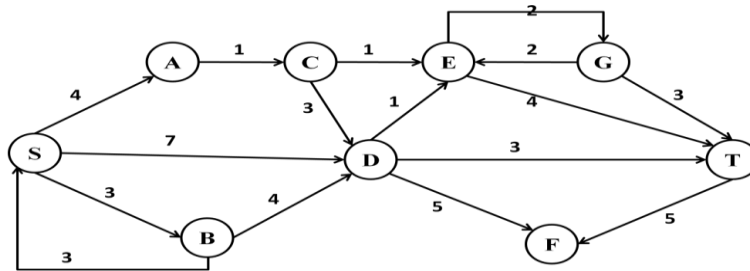
Visited	A	B	C	D	E
{A}	0	4	∞	8	∞
{A,B}	0	4	7	8	∞
{A,B,C}	0	4	7	8	∞
{A,B,C,D}	0	4	7	8	15



Therefore, shortest path is



Qn) Find the shortest path from s to all other vertices in the following graph using Dijkstra's Algorithm

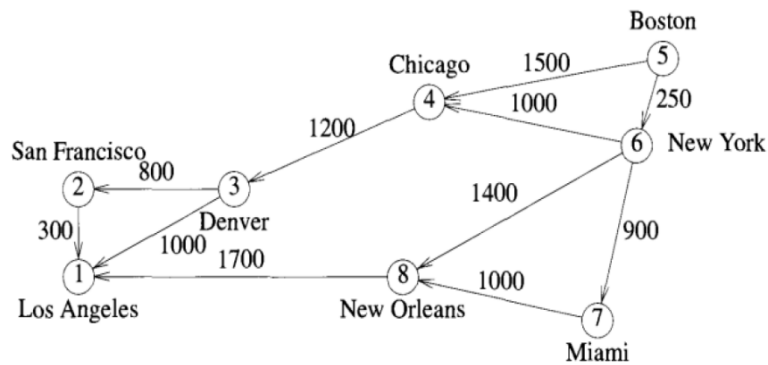


Ans)

	S	A	B	C	D	E	F	G	T
S	0	α	α	α	α	α	α	α	α
B		4	3	α	7	α	α	α	α
A		4		α	7	α	α	α	α
C				5	7	α	α	α	α
E					7	6	α	α	α
D					7		α	8	10
G							12	8	10
T							12		10
F							12		

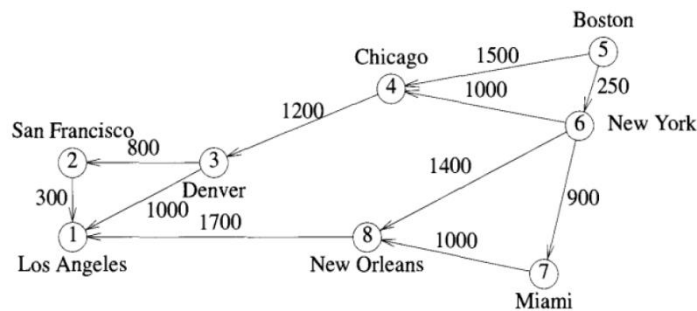
	Shortest Path	Shortest Distance
S-A	S-A	4
S-B	S-B	3
S-C	S-A-C	5
S-D	S-D	7
S-E	S-A-C-E	6
S-F	S-D-F	12
S-G	S-A-C-E-G	8
S-T	S-A-C-E-T	10

Qn)



(a) Digraph

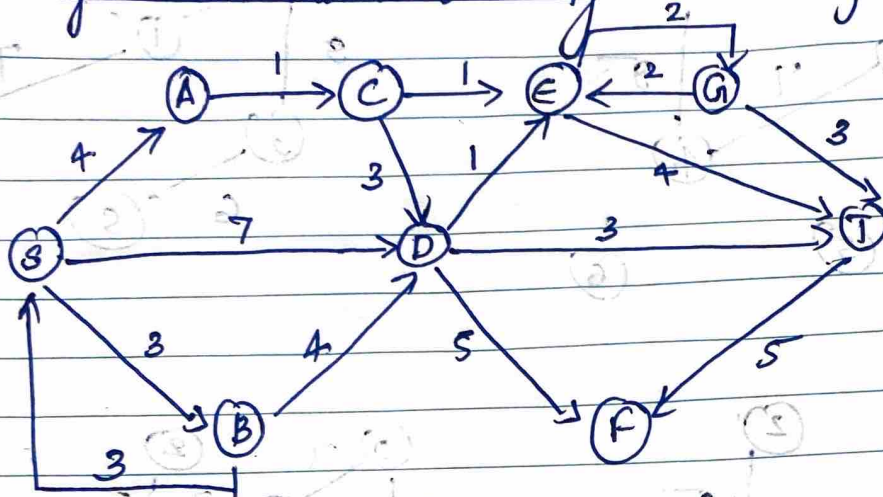
Ans)



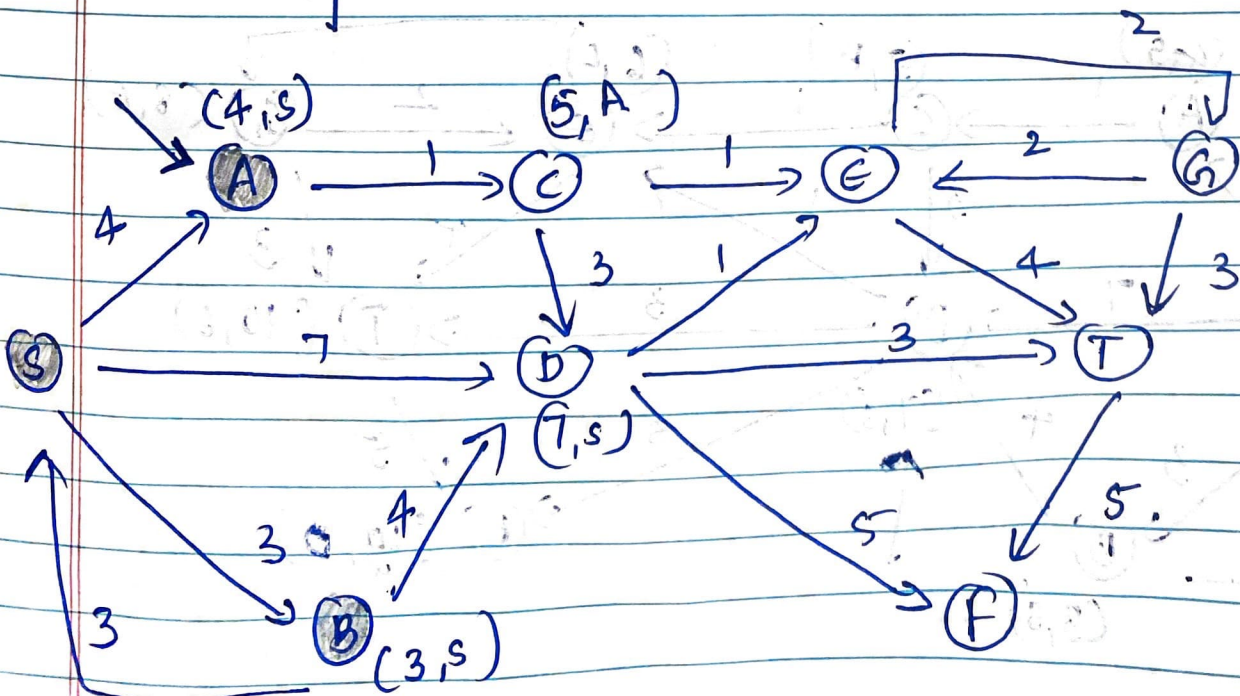
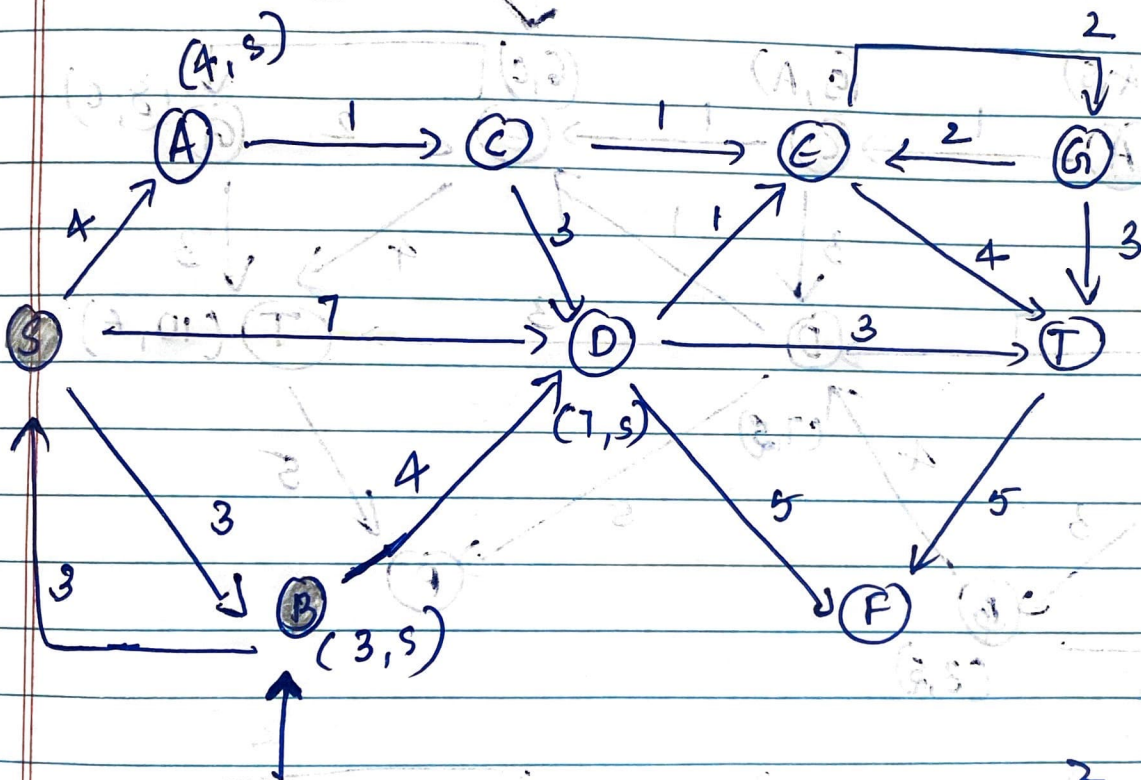
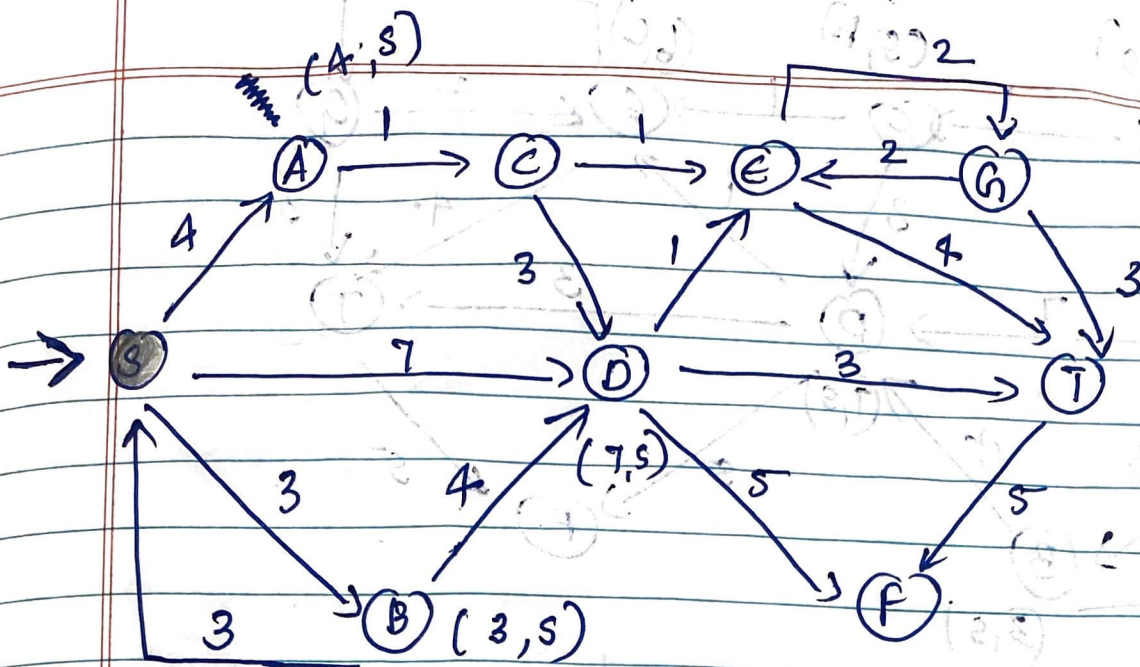
Iteration	S	Vertex selected	Distance							
			LA	SF	DEN	CHI	BOST	NY	MIA	NO
			[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
Initial	--	----	+∞	+∞	+∞	1500	0	250	+∞	+∞
1	{5}	6	+∞	+∞	+∞	1250	0	250	1150	1650
2	{5,6}	7	+∞	+∞	+∞	1250	0	250	1150	1650
3	{5,6,7}	4	+∞	+∞	2450	1250	0	250	1150	1650
4	{5,6,7,4}	8	3350	+∞	2450	1250	0	250	1150	1650
5	{5,6,7,4,8}	3	3350	3250	2450	1250	0	250	1150	1650
6	{5,6,7,4,8,3}	2	3350	3250	2450	1250	0	250	1150	1650

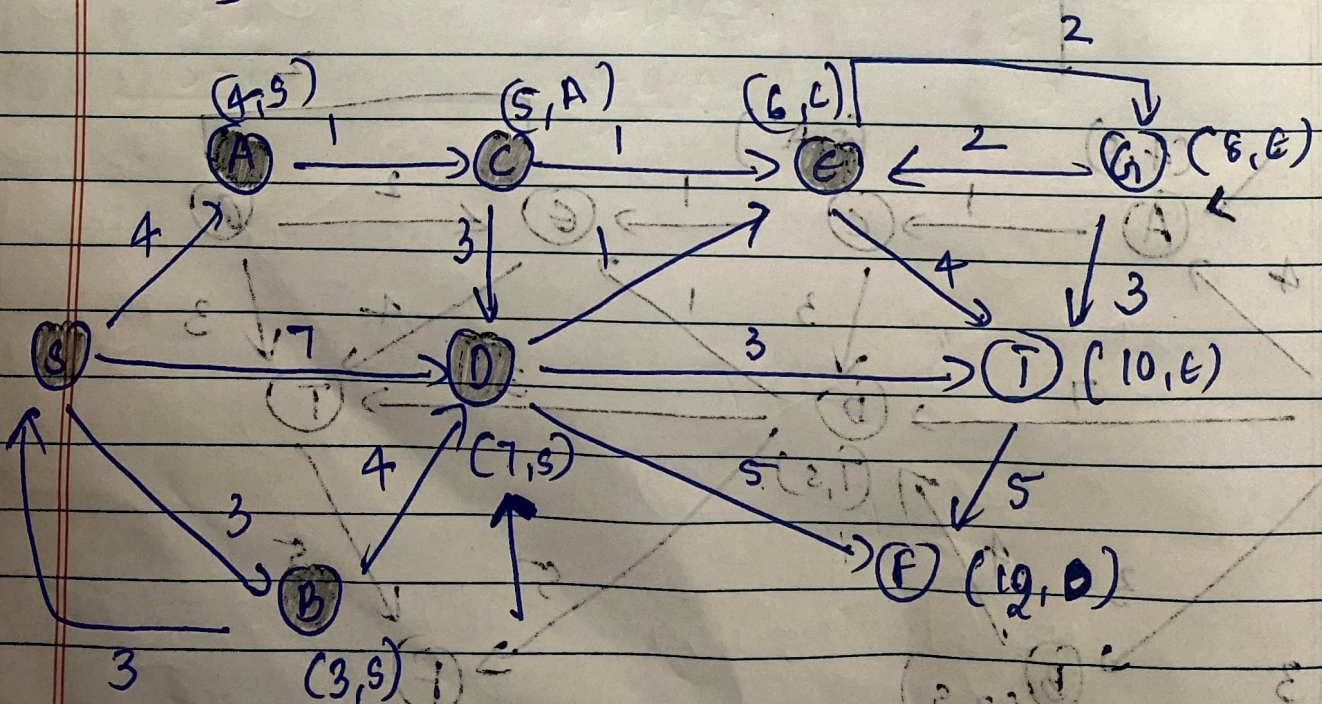
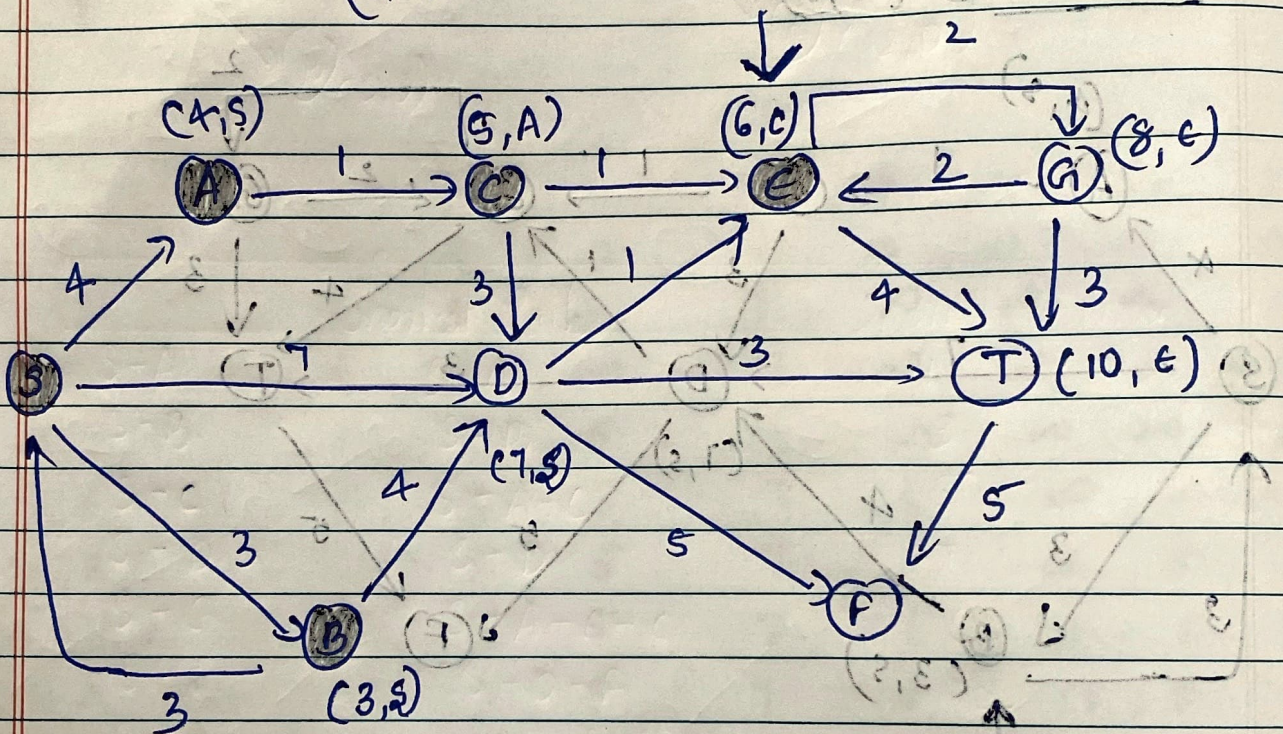
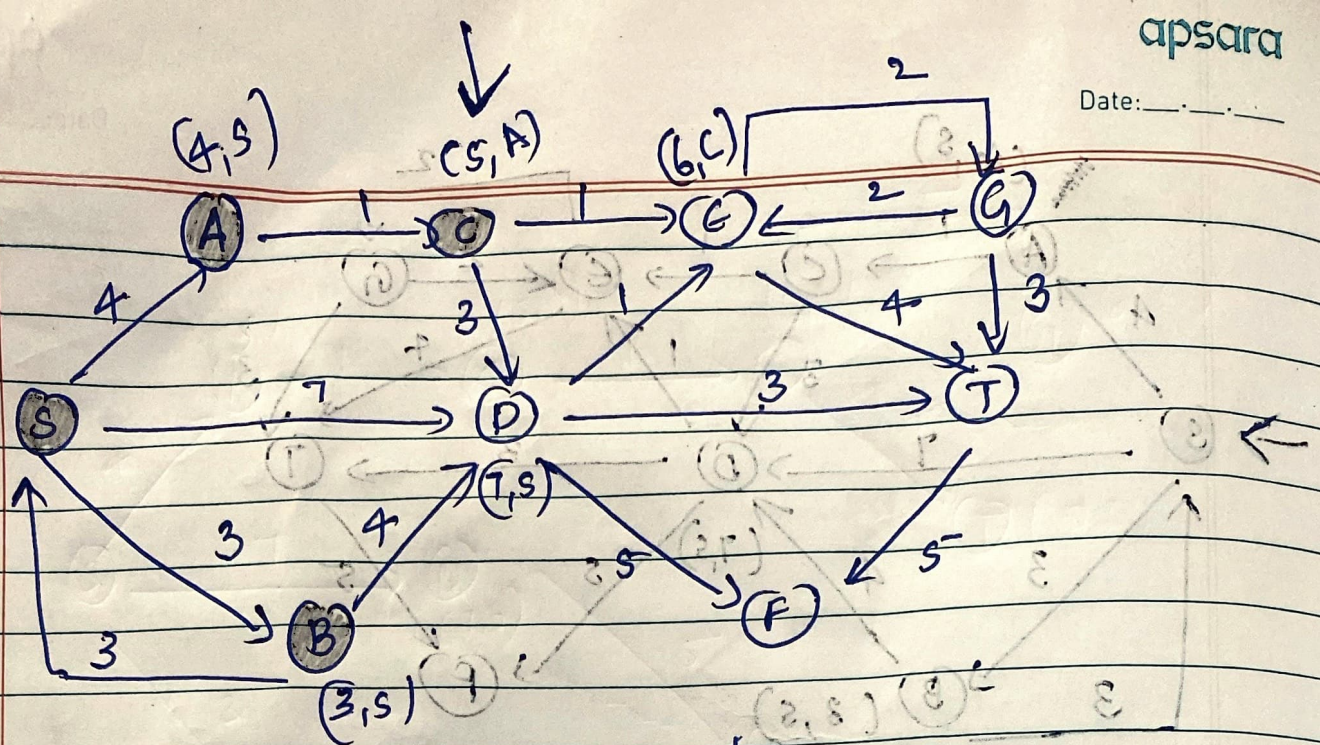
Single Source Shortest Path Algorithm - Dijkstra's Algorithm

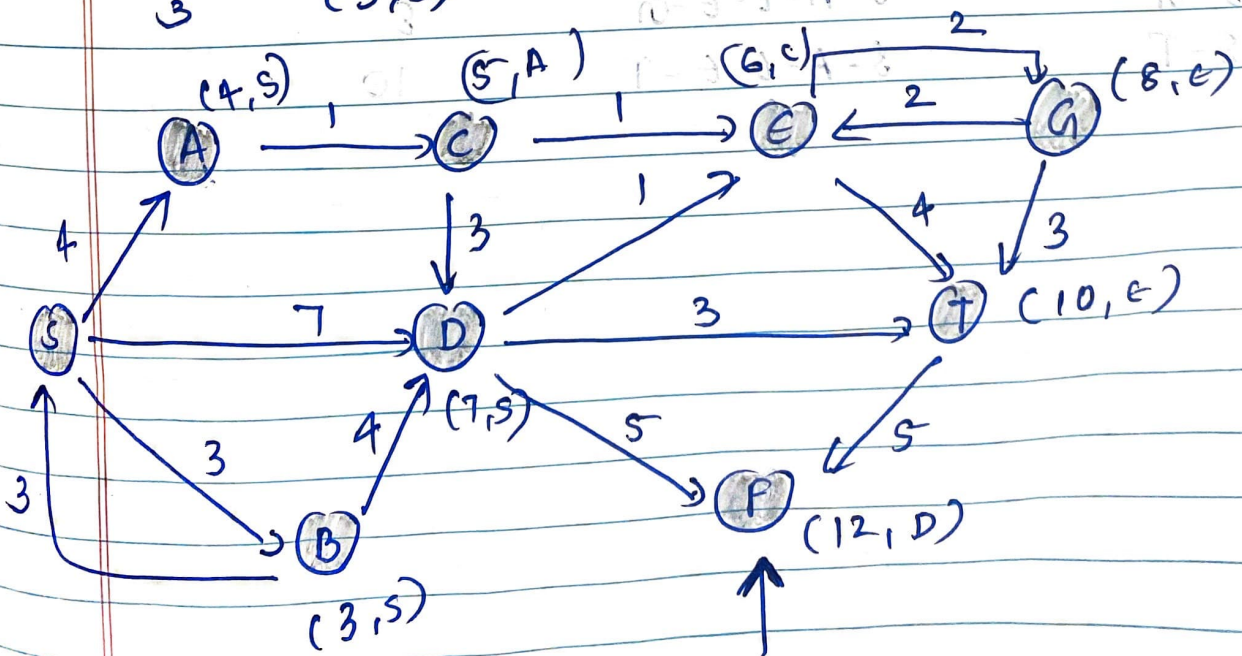
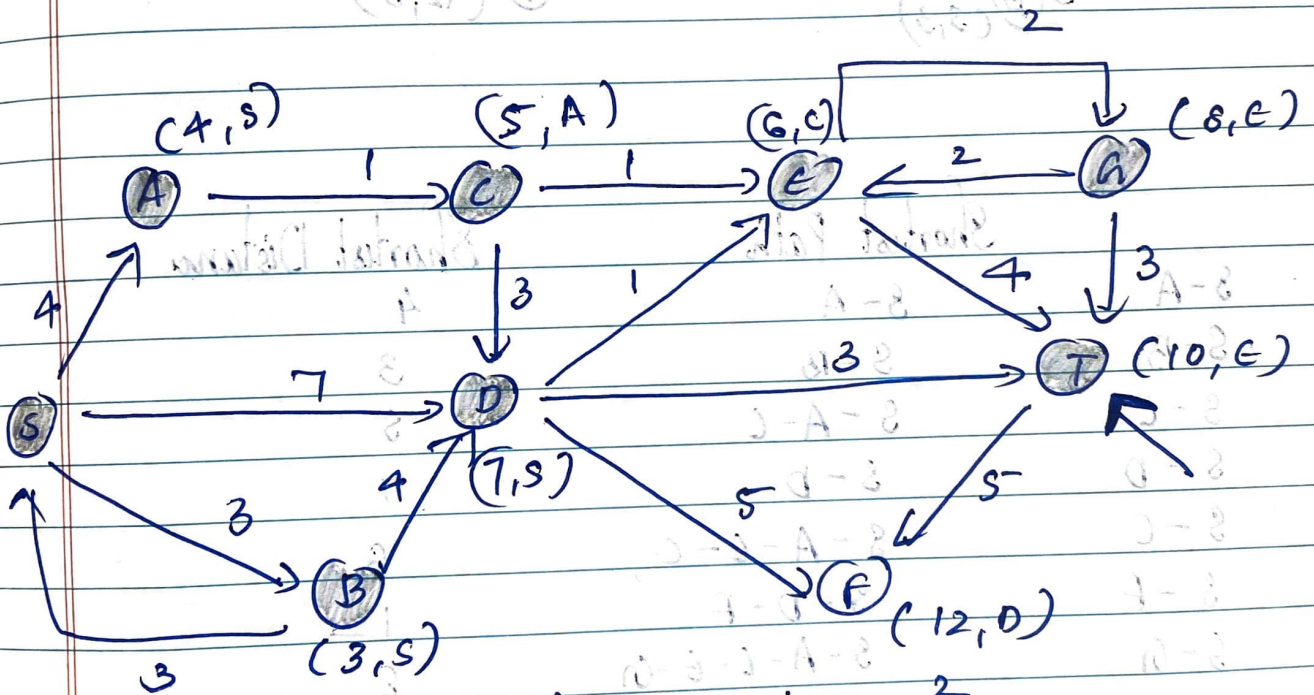
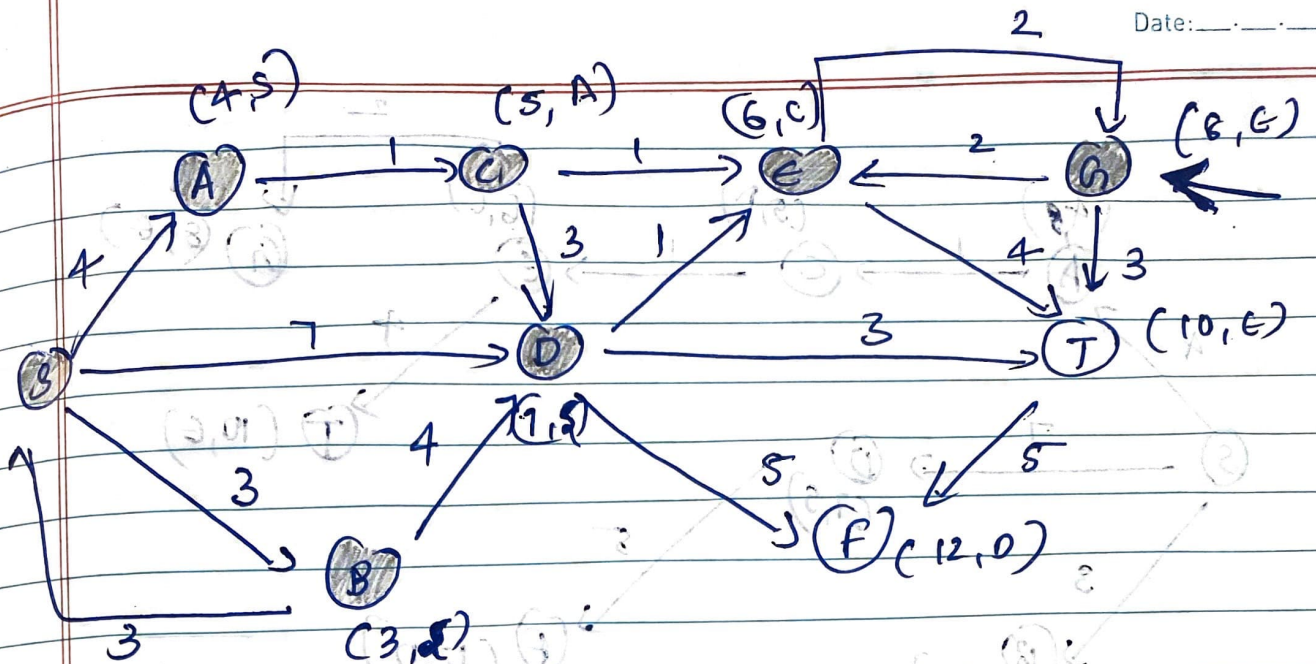
a.

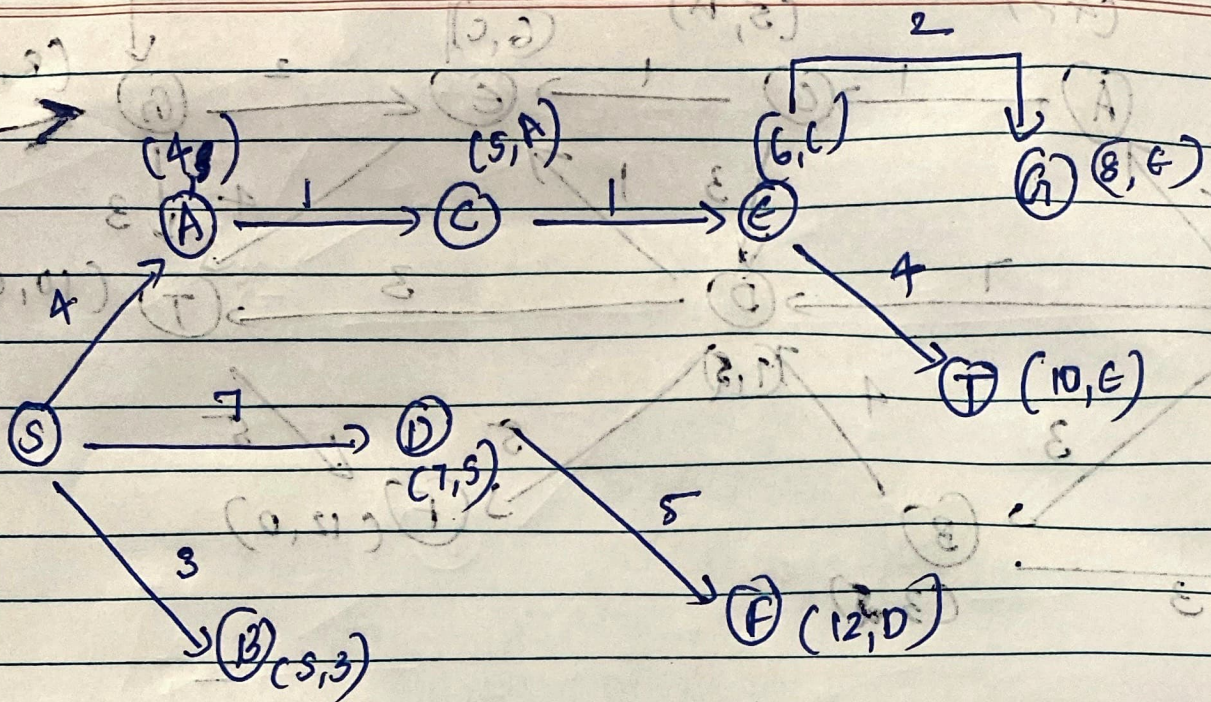


Visited	A	B	C	D	E	F	G	S	T
Initially	∞	∞	∞	∞	∞	∞	∞	0	∞
{S}	4	3	∞	7	∞	∞	∞	0	∞
{S, B}	4	3	∞	7	∞	∞	∞	0	∞
{S, B, A}	4	3	5	7	∞	∞	∞	0	∞
{S, B, A, C}	4	3	5	7	6	∞	∞	0	∞
{S, B, A, C, E}	4	3	5	7	6	∞	8	0	10
{S, B, A, C, E, D}	4	3	5	7	6	12	8	0	10
{S, B, A, C, E, D, G}	4	3	5	7	6	12	8	0	10
{S, B, A, C, E, D, G, T}	4	3	5	7	6	12	8	0	10
{S, B, A, C, E, D, G, T, F}	4	3	5	7	6	12	8	0	10









	Shortest Path	Shortest Distance
S-A	S-A	4
S-B	S-B	3
S-C	S-A-C	5
S-D	S-D	7
S-E	S-A-C-E	6
S-F	S-D-F	12
S-G	S-A-C-E-G	8
S-T	S-A-C-E-T	10